



Discrete Optimization

Sequencing with learning, forgetting and task similarity

Shuling Xu^a, Fulong Xie^b, Nicholas G. Hall^{c,*,*}^a Jiangnan University, Wuxi, Jiangsu, China^b Hefei University of Technology, Hefei, Anhui, China^c The Ohio State University, Columbus, OH, 43210-1144, USA

ARTICLE INFO

Keywords:

Human–machine workplace

Scheduling

Learning and forgetting

Algorithms and heuristics

ABSTRACT

In human–machine workplaces, employee skills change over time due to learning and forgetting effects, which greatly affects efficiency. We model these effects within a simple production system. When a job is processed, due to learning the next job of the same type requires less processing time. Meanwhile, jobs of other types are forgotten, hence their future processing times increase. In our work, the amount of learning and forgetting depends on the full matrix similarity of the two job types and on the size of the jobs processed. We describe a dynamic programming algorithm that minimizes the makespan optimally. More generally, the learning level of a job type has an upper limit above which further learning is lost, and a lower limit below which further forgetting is saved. For this more general problem, we adapt our dynamic programming algorithm to provide tight lower bounds that validate the performance of simple heuristic approaches and genetic algorithms. A computational study demonstrates that these procedures routinely deliver optimal, or very close to optimal, solutions for up to eight job types and 80 jobs. Our work provides what are apparently the first effective procedures for optimization of large-scale production schedules with learning and forgetting effects defined by full matrix similarity. This identifies a critical opportunity for human–machine collaboration to improve productivity and support operational excellence.

1. Introduction

Modern manufacturing systems, such as intelligent manufacturing systems, integrate the capabilities of machines, humans, and processes to improve production efficiency (Asghari et al., 2024). These “human–machine” systems enable workers to perform a variety of functionally different operations to meet varying demands and achieve operational flexibility. The flexibility provided by these systems is recognized as contributing greatly to production system efficiency (Frank et al., 2019; Geurtsen et al., 2023). However, the implementation of this flexibility poses challenges for human–machine manufacturing systems. Issues such as learning and forgetting effects (Ostermeier, 2020; Heuser & Tauer, 2023) introduce variability in the time required to perform different tasks when workers switch between different tasks. Since production time is a central measure of manufacturing performance, an improvement in efficiency resulting from optimal scheduling provides potentially huge benefits (Borgonjon & Maenhout, 2024). Our work studies learning and forgetting effects within a manufacturing system that produces multiple types of products.

Within the scheduling literature, there are several studies that consider learning effects which result in improved performance of the schedule as processing continues. For example, if two similar jobs of

the same type are processed sequentially in the schedule, the second job is processed more efficiently, resulting in a shorter processing time. This represents learning, as first explored by Wright (1936). However, when processing different types of products, machines need to adapt frequently to different types of tasks. These adjustments cause machines to forget the optimized settings and experience gained from previous tasks, similar to the phenomena of multitask learning and catastrophic forgetting described by Kar et al. (2022) and Heuser and Tauer (2023). In addition interruptions, such as task-switching or breaks taken by workers, may cause interference in recalling task-specific setup procedures, ultimately leading to forgetting (Jaber et al., 2021; Borgonjon & Maenhout, 2024; Ostermeier & Deuse, 2024). In this work, we emphasize that the speed of forgetting increases with the *pairwise dissimilarity* of the type of task being processed and that of any waiting task.

Our work is motivated by practical considerations in human–machine cooperative scheduling, with a particular focus on problems where the number of job types is *fixed* and typically small. In manufacturing processes, jobs often share common attributes, such as color in a painting shop or tool compatibility in flexible manufacturing systems. Van Der

* Corresponding author.

E-mail address: hall.33@osu.edu (N.G. Hall).<https://doi.org/10.1016/j.ejor.2025.03.002>

Received 15 August 2024; Accepted 3 March 2025

Available online 5 April 2025

0377-2217/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Veen and Zhang (1996) address this scenario by providing efficient algorithms for single-machine and multi-machine problems with a fixed number of job types, to minimize makespan. Real world applications are studied by Shabtay and Karhi (2012) and Karhi and Shabtay (2014), focusing on scheduling problems with two job types on single or identical parallel machines. Also, Kouteck and Zink (2020) analyze problem complexity and develop efficient algorithms for scheduling a small, fixed number of job types on related and unrelated parallel machines. These studies highlight the practical relevance of addressing scheduling challenges associated with a fixed and typically small number of job types.

The assumption of a *fixed job sequence* within a job type is motivated by technological or managerial decisions, and is a well-studied research issue in scheduling. This assumption is also driven by customer fairness considerations, such as the First-Come-First-Served rule. Lin and Hwang (2011) and Hwang and Lin (2011) develop algorithms to minimize makespan and total completion time under fixed job sequences. Further, Hwang et al. (2012, 2014) demonstrate the effectiveness of fixed job sequences in optimizing two-machine flowshops and assembly operations. These studies highlight the practical relevance of addressing the scheduling challenges associated with fixed job sequences within a job type in various real-world applications. This is the environment which our work studies.

The main contributions of this study are as follows. We consider a problem of scheduling multiple job types to minimize the makespan, in the presence of learning effects and full matrix forgetting effects, as well as classical position-based learning. We describe a dynamic programming algorithm that minimizes the makespan optimally. A more general problem with upper and lower limits on learning is then considered. We adapt the algorithm to provide tight lower bounds that validate the performance of simple heuristic approaches and genetic algorithms. A computational study reveals that these procedures routinely deliver optimal, or very close to optimal, solutions for up to eight job types and 80 jobs. Our work provides what are apparently the first effective procedures for optimization of large-scale production schedules with learning and forgetting effects defined by *full matrix similarity*.

This paper is organized as follows. Section 2 reviews previous work on scheduling with learning and forgetting. Section 3 provides some notation, introduces our model, and describes a dynamic programming algorithm that solves the scheduling problem with upper and lower bounds on learning optimally, but is not computationally efficient for large problems. Section 4 describes easily implemented heuristic procedures that find close to optimal feasible solutions. In Section 5, we improve solutions found by these heuristics using a customized genetic algorithm. Section 6 describes a procedure for generating tight lower bounds on optimal solution value. In Section 7, we report the results of our computational study, including a detailed sensitivity analysis. Section 8 contains a summary of our work and suggestions for future research.

2. Literature review

Section 2.1 discusses work on learning and forgetting curves. Section 2.2 reviews work on the job type similarity, and how it affects learning and forgetting rates. Section 2.3 discusses solution approaches to scheduling problems with both learning and forgetting effects.

2.1. Learning and forgetting curves

Both learning and forgetting effects have been studied for many years. Wright (1936) models learning effects as a log-linear function of the number of units produced, to obtain a prediction of processing times needed. Bailey (1989) models forgetting of a processed task as a linear function of the product of the amount learned and the logarithm of elapsed time. Reviews of the related literature appear in Geurtsen

et al. (2023). Learning and forgetting curves are combined by Jaber (2006), who allow complete forgetting, where the production time of a unit product returns to its original level.

Several studies consider learning and forgetting for a diverse range of applications. For competitive marketplaces, Haunschild et al. (2015) develop a model that explains why firms cycle through periods of learning and other periods of forgetting. In order to assess the impact of both learning and forgetting on long term competitiveness and a firm's profitability, Kogan et al. (2017) model an experience accumulation process with depreciation and study the decisions of two competing firms that produce fully substitutable products. In healthcare, Ramdas et al. (2018) examine learning and forgetting in hip replacement surgery as a function of a surgeon's experience with specific surgical devices and the time between their repeated uses. In service industries, Afshari et al. (2019) study the effects of learning and forgetting on the feasibility of adopting additive manufacturing in supply chains. To summarize this literature, learning and forgetting effects are important determinants of efficiency for a wide variety of systems where humans and machines interact.

2.2. Effect of type similarity on forgetting

Forgetting of work of a particular type occurs through the processing of work of another type. Increased demand for customization is forcing companies to reduce batch sizes continuously during production which leads to more interruptions and thus more forgetting (Korytkowski, 2017; Howick & Eden, 2007). Here, the similarity between the job type being processed and the other job types naturally influences the latter's forgetting rates.

Hoedt (2019) develop a model that incorporates both learning and forgetting effects. Their model is used predictively, to estimate production times for different jobs, and performs well when tested against real world data, but no optimization of schedules is attempted. Heuser and Tauer (2023) study a problem where jobs are classified by categories. For the purpose of estimating forgetting rates, jobs are either in the same category or not, without consideration of pairwise similarity. They also model upper and lower bounds on learning, a feature that we include in our work. They describe three constructive heuristic and evaluate their performance for problem instances with up to 100 jobs. By contrast with their work, we describe an optimal algorithm for scheduling with learning and forgetting that is influenced by similarity that is defined both by the job being processed and the waiting job(s), which we define as full matrix similarity.

The previous work that is closest to ours is Pan et al. (2014). They consider a single machine problem with learning and forgetting effects, random breakdowns, and preventive maintenance. Sequencing and preventive maintenance decisions are used to minimize the expected makespan heuristically with a genetic algorithm. As in our work, the amount of forgetting of a waiting job depends on its similarity to the job being processed. An example with nine jobs of three types is tested. Their results provide comparisons of performance across different heuristic designs and sensitivity analysis to various parameter changes. They do not attempt the optimal or heuristic solution of instances with more than nine jobs. We are able to find routinely close to optimal solutions for up to eight job types with 80 jobs.

2.3. Scheduling with learning and forgetting

Many learning and forgetting models described are successful in making predictions about manufacturing performance, on the basis of Wright's learning curve, but do not optimize schedule performance. However, Teyarachakul et al. (2011) assume the rate of forgetting is related to the length of the interruption, and develop a forward recursive algorithm to solve the production batch sizing problem with learning and forgetting in processing times. Mustu and Eren (2021)

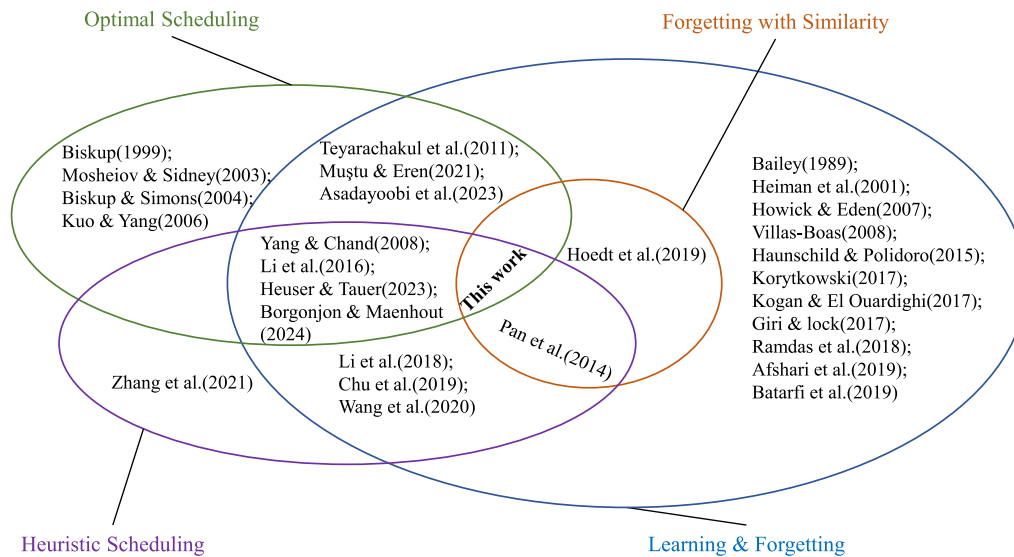


Fig. 1. Scheduling with learning, forgetting, and similarity.

propose an integer non-linear programming model and a dynamic programming model to solve a single machine scheduling problem, where the forgetting effect depends on the duration of setups and the amount of learning before the setup. Asadayoobi et al. (2023) address stochastic bi-objective task allocation and scheduling problems for mission workers subject to the joint effect of learning-forgetting, fatigue-recovery, and stress-recovery processes. A Markov Decision Process is used to model this stochastic problem. None of the above works consider the effect of task similarity on forgetting rates.

Several papers propose heuristics for scheduling problems. Yang and Chand (2008) propose a branch-and-bound algorithm and an efficient heuristic for scheduling families of jobs on a single machine to minimize the total completion time. Zhang et al. (2021) observe that the learning effect stops when it reaches a given highest skill level, a feature that we also model. Chu et al. (2019) propose an adaptive memetic differential search algorithm to solve the problem of worker assignment in cellular manufacturing systems (CMS). Wang et al. (2020) describe a learning and forgetting model of integrated cell formation and task scheduling for a CMS and propose an improved bacterial foraging algorithm to minimize makespan.

Our work describes a scheduling model with learning and full-matrix forgetting effects, as well as position-based learning. Fig. 1 shows the most closely related works in the literature discussed above. Our work is uniquely located in the quadruple intersection of learning and forgetting, forgetting with full matrix similarity, and both optimal and heuristic scheduling.

3. Model and algorithm

We consider a human-machine production environment that requires the scheduling of a given set of jobs on a single processor. There are m types of jobs, and it is necessary to schedule n_j jobs of each Type j nonpreemptively, i.e. without interruption between the start and completion time of the same job, for $j = 1, \dots, m$. Let $n = \sum_{j=1}^m n_j$. Within each job type, the sequence of jobs is fixed. Also within each job type, the nominal processing times of the jobs, i.e. their processing times unadjusted for learning and forgetting, varies between them. Actual job processing times are subject to learning and forgetting based on their types, but otherwise deterministic. The actual processing time of a job, for example of Type j , is reduced whenever another job of Type j is processed, due to *learning*. However, when the processing of jobs of Type j is interrupted by the processing of jobs of another Type $i \neq j$, the processing time of the next job of Type j is increased due to *forgetting*.

The rate of forgetting is a function of the *similarity* of job Types i and j ; if the two job types are more similar, then the rate of forgetting is lower. The decision required is the sequencing of all the jobs. We consider the classical scheduling objective of minimizing the overall schedule length, i.e. makespan. This is Problem Scheduling with Learning and Forgetting (Problem SLF).

The minimization of makespan is a central and classical objective in scheduling theory and practice. Given the focus of our work, it could also be relevant to consider end-of-schedule learning levels. However, for three reasons, we focus on the makespan. First, our work is motivated by practical production scenarios where jobs are processed in a single batch that is independent of later batches, which makes the final learning levels at the end of the batch irrelevant. Second, in cases where there is substantial idle time between consecutive batches, the downtime until the start of the next batch is sufficiently long for all learning levels to revert to their original values. Finally, optimization over the final learning states would be problematic because there would be tradeoffs involving the makespan and the final states, but those tradeoffs are ill-defined without information about the next batch of jobs.

Following the classical scheduling literature, we assume that all data is integer. The Type j jobs have nominal processing times $p_{j,1}, \dots, p_{j,n_j}$, for $j = 1, \dots, m$, where we let $P_{j,k} = \sum_{i=1}^k p_{j,i}$ and $P_j = \sum_{i=1}^{n_j} p_{j,i}$. During the schedule, all jobs of Type j have the same *learning level* v_j^t , which is initially v_j^0 but increases due to learning and decreases due to forgetting. However, the learning level is bounded by $L_j \leq v_j \leq U_j$. If the learning level of Type j is at or close to U_j and further scheduling of jobs of that type occurs, then some *lost learning* may occur, since the job is fully learned at $v_j = U_j$. Symmetrically, if the learning level of Type j is at or close to L_j and further scheduling of jobs of other types occurs, then some *saved forgetting* may occur, since all learning has been lost at $v_j = L_j$.

We discuss the modeling of learning and the consequent reduction in a job type's processing factor. We adopt the classic learning function of Wright (1936), but in a discretized form. The purpose of discretizing the learning levels is to improve the computational tractability of our dynamic programming algorithms described below. In Section 7.4 below, we verify that discretization of the learning levels does not result in any significant loss of solution accuracy. The processing factor in Wright's work is $x_j^{l(j)}$, where x_j is the number of Type j jobs processed and $l(j)$ is a learning parameter specific to job Type j . However in our work, unlike Wright's, the nominal processing times of the jobs vary within a single job type. Therefore, it is natural to expect more learning

during the processing time of a longer job of the same type. For this reason, we generalize the learning function of Wright (1936) to define a change in learning level of $l_j p_{j,x_j}$ for the first through x_j th jobs of Type j , in the absence of any lost learning at $v_j = U_j$ and any forgetting. An example of the learning-forgetting process is given in Appendix A.1.

We next discuss the modeling of forgetting for a job type. Every time a job of Type $j \neq i$ is processed, the learning level of Type i jobs is decreased due to forgetting. Once again, the nominal processing times of the Type j jobs vary, and it is natural to expect a larger forgetting effect for the jobs of Type i when a Type j job with a longer processing time is being processed. Thus, symmetrically with the above learning effect, we model the cumulative effect on the learning level of any job of Type i from the scheduling of the first through x_j th jobs of Type j as $f_{ji} p_{j,x_j}$, where f_{ji} is a forgetting parameter that depends on the similarity of the two job types, in the absence of any saved forgetting at $v_j = L_j$ and any learning. Further, the scheduling of the first x_j jobs of multiple job types $j \neq i$ contributes to the forgetting of Type i jobs, hence we write the total forgetting effect as $\sum_{1 \leq j \neq i \leq m} f_{ji} p_{j,x_j}$. Following Pan et al. (2014), we define $f_{ji} = 1 - r_{ji}$, where r_{ji} denotes the similarity of the Type j of the job being processed and the Type i of a job waiting for processing.

Combining the learning and forgetting effects described in the two paragraphs above, we assume that x_i jobs of Type i have been scheduled, for $i = 1, \dots, m$. Then, the learning level of the jobs of any Type j , in the absence of lost learning and saved forgetting, is: $v_j = v_j^0 + \sum_{k=1}^{x_j} l_j p_{j,k} - \sum_{1 \leq i \neq j \leq m} \sum_{k=1}^{x_i} f_{ij} p_{i,k}$. We let $\phi_j(v'_j, x_j) > 0$ denote the actual processing time of the x_j th job of Type j with learning level v'_j adjusted for previous learning and forgetting, where $\phi_j(\cdot, \cdot)$ is nonincreasing in its first argument and nondecreasing in its second argument. We now provide additional details.

Various learning and forgetting functions have been modeled in different production processes, influenced by factors such as worker efficiency, machine performance, and job properties, for example Pei et al. (2023). We now describe our model of $\phi_j(v'_j, x_j)$. Following the literature on human-machine processing (Akbari Arekamari & Ghasemi Namaghi, 2023; Asghari et al., 2024), we allow for adjustments to the nominal processing time based on (a) learning and forgetting effects, and (b) the position of the job in the processing sequence. The first adjustment is dependent on the learning level, which is machine-dependent and results from learning and forgetting due to the effects of previously scheduled jobs. This approach is similar to that of Pan et al. (2014) in considering the sum of learning and forgetting effects for the single-machine group scheduling problem, and Heuser and Tauer (2023) in considering category-based learning and forgetting effects for a single-machine scheduling problem. The second adjustment depends on the position in the sequence, which indicates that individuals enhance their performance based on their experiences in specific positions. Yin et al. (2013) explore this within single-machine scheduling, showing that the processing time for a job in a group is affected by the cumulative processing times of prior jobs and its sequence position. Following Borgonjon and Maenhout (2024) in using a discretization of the learning level to model learning and forgetting effects, we define

$$\phi_j(v'_j, x_j) = p_{j,x_j} * (v_j^0/v'_j)^{\alpha_{1j}} * (\sum_{i=1}^m x_i)^{\alpha_{2j}}, \quad (1)$$

where $\alpha_{1j} \geq 0$ and typically close to 1, and $\alpha_{2j} < 0$ and typically close to 0. In (1), the first term p_{j,x_j} is the nominal processing time of the x_j th job of Type j . Based on learning level adjustments from the scheduling of the earlier jobs of Type j which provide learning, and of the other job types which induce forgetting, the second term $(v_j^0/v'_j)^{\alpha_{1j}}$ adjusts the actual processing time. We note that if job $(j, 1)$ is the first job scheduled, hence $v'_j = v_j^0$, and $\sum_{i=1}^m x_i = 1$, then the actual processing time is identical to the nominal processing time. In general, we do not assume that the initial processing time of a job is equal to its nominal processing time. The third term identifies job (j, x_j) as the $\sum_{i=1}^m x_i$ th job

to be processed and, based on that position, adjusts its processing time for human learning effects.

We now describe a dynamic programming algorithm, SLFDP, that constructs an optimal schedule for the given jobs. The algorithm keeps track of the learning level of each job type, as well as the number of jobs of that type so far scheduled. Since all learning is caused by the scheduling of jobs of the same type, and all forgetting is caused by the scheduling of the jobs of other types, this information is sufficient to update the learning levels of all the job types.

Algorithm 1: SLFDP

Input : The learning effects l_j , the forgetting effects f_{ij} , the initial learning levels v_j^0 , the nominal processing times of the job p_{j,n_j} , the lower bounds on learning L_j , and the upper bounds on learning U_j

Output: The minimum makespan $g(v_1, \dots, v_m; x_1, \dots, x_m)$

1 **Value Function:** $g(v_1, \dots, v_m; x_1, \dots, x_m)$ = the minimum makespan for processing jobs $1, \dots, x_j$ of Type j , where the resulting learning level of Type j is $L_j \leq v_j \leq U_j$, for $j = 1, \dots, m$;

2 **Boundary Condition:** $g(v_1^0, \dots, v_m^0; 0, \dots, 0) = 0$;

3 **Optimal Solution Value:**

$$L_j \leq v_j \leq U_j, j=1, \dots, m \{g(v_1, \dots, v_m; n_1, \dots, n_m)\};$$

4 **Recurrence Relations:**

$$g(v_1, \dots, v_m; x_1, \dots, x_m) = \min_{1 \leq j \leq m, L_j \leq v'_j \leq U_j} \{\phi_j(v'_j, x_j) +$$

$$g(v'_1, \dots, v'_m; x_1, \dots, x_{j-1}, x_j - 1, x_{j+1}, \dots, x_m)\}, \text{ where}$$

$$v_j = \min\{v'_j + l_j p_{j,x_j}, U_j\}, \text{ and}$$

$$v_i = \max\{v'_i - f_{ji} p_{j,x_j}, L_i\}, 1 \leq i \neq j \leq m.$$

Algorithm SLFDP begins with no jobs scheduled. The learning level of job j is initially v_j^0 , where $L_j \leq v_j^0 \leq U_j$, for $j = 1, \dots, m$. The recurrence relation repeatedly chooses a single job (j, x_j) , which is the x_j th job of Type j , to add to the schedule. This results in an increase in the learning level of the jobs of Type j from v'_j to $v'_j + l_j p_{j,x_j}$ or U_j , whichever is less. If $v'_j + l_j p_{j,x_j} > U_j$, then the jobs of Type j have been fully learned at U_j , and therefore some potential learning effect from the scheduling of job (j, x_j) is lost. Meanwhile, the scheduling of job (j, x_j) results in some forgetting for jobs of Type i , $1 \leq i \neq j \leq m$. Hence, the learning level of job i decreases from v'_i to $v'_i - f_{ji} p_{j,x_j}$ or L_i , whichever is greater. If $v'_i - f_{ji} p_{j,x_j} < L_i$, then the jobs of Type i have been fully unlearned or forgotten at L_i , and therefore some potential forgetting effect from the scheduling of job (j, x_j) is saved. The value function updates the makespan of the schedule by the processing time $\phi_j(v'_j, x_j)$ of job (j, x_j) , as adjusted for the current learning level of Type j . The algorithm terminates when all jobs have been scheduled, at various possible configurations of feasible learning levels for all the job types that depend on the sequence chosen and consequently on lost learning and saved forgetting.

Theorem 1. Algorithm SLFDP finds an optimal schedule for Problem SLF in

$$O(m \prod_{j=1}^m [U_j - L_j]^2 n_j) \text{ time.}$$

Proof. Observe that the processing time of any job of any type is determined by its nominal processing time and the learning level of that type. Hence, the state space $(v_1, \dots, v_m; x_1, \dots, x_m)$ completely characterizes the information that is needed to compute the remaining cost to complete the schedule. Therefore, the recurrence relation compares the cost of all possible schedules, and finds an optimal schedule.

We observe that $L_j \leq v_j \leq U_j$ and $0 \leq x_j \leq n_j$, for $j = 1, \dots, m$. Further, the recurrence relation requires $O(m \prod_{j=1}^m [U_j - L_j])$ comparisons. The computational requirement of Algorithm SLFDP is $O(\prod_{j=1}^m [U_j - L_j] n_j \cdot m \prod_{j=1}^m [U_j - L_j]) = O(m \prod_{j=1}^m [U_j - L_j]^2 n_j)$ time. $\square$

We provide a numerical example of Algorithm SLFDP in Appendix A.2.

4. Heuristics

Due to the long running time of Algorithm SLFDP for large instances, we propose several “greedy” heuristics, GH1 through GH4, to obtain a feasible schedule of the jobs with a near-optimal makespan. These heuristics are designed to balance computational feasibility with solution quality, making them suitable for practical application for large-scale scheduling problems, as well as easy to implement in practice as standalone solution procedures. The descriptions refer to *learning increments* and *forgetting decrements* to the current learning levels of job types.

4.1. GH1

For all job types with unscheduled jobs, calculate the learning increment divided by the average forgetting decrement, and then choose the job with maximum ratio to schedule first. If the forgetting unit of all types is 0, a job type with largest learning unit is chosen.

Algorithm 2: GH1

Data: $l_j, f_{ij}, v_j^0, p_{j,x_j}, L_j, U_j$, job types $m, n = \sum_{j=1}^m n_j$
Result: Minimum makespan $g(v_1, \dots, v_m; x_1, \dots, x_m)$

```

1 Initialize  $P \leftarrow \emptyset; J \leftarrow \text{unscheduled jobs}$ ;
2 while  $J \neq \emptyset$  do
3   for each  $(j, x_j) \in J$  do
4      $\delta_l^+(j, x_j) \leftarrow \min\{l_j p_{j,x_j}, U_j - v_j\}$ ;
5      $\delta_f^-(j, x_j) \leftarrow (\sum_{h \neq j}^m \min\{f_{jh} p_{j,x_j}, v_h - L_h\}) / (m - 1)$ ;
6   end
7    $S \leftarrow \text{None}$ ;
8   if  $\min_{(j,x_j) \in J} \{\delta_f^-(j, x_j)\} = 0$  then
9      $S \leftarrow \arg\max_{(j,x_j) \in J} \{\delta_l^+(j, x_j)\}$ ;
10  end
11  else
12     $S \leftarrow \arg\max_{(j,x_j) \in J} \{\delta_l^+(j, x_j) / \delta_f^-(j, x_j)\}$ ;
13  end
14  Process  $S$ ; Add  $S$  to  $P$ ; Remove  $S$  from  $J$ ; Update
     $g(v_1, \dots, v_m; x_1, \dots, x_m)$ ;
15 end
16 return  $g(v_1, \dots, v_m; x_1, \dots, x_m)$ ;

```

4.2. GH2

For all job types with unscheduled jobs, calculate the ratio of the learning increment of the next job of the same type to the total forgetting decrement of the jobs of the other types, and select a type with largest ratio. When there is only one job left of all types, choose a job type with smallest forgetting decrement.

4.3. GH3

Similar to GH2, the learning increment is calculated by considering all remaining jobs of the same type, excluding the job currently scheduled for processing.

4.4. GH4

For all job types with unscheduled jobs, calculate the ratio of the learning increment of next jobs of each type, to the total forgetting decrement of the jobs of the other types, and select a type with largest ratio. This approach prioritizes scenarios where the learning effect outweighs the total forgetting impact. If the ratios are equal, select one with largest learning increment. If the forgetting increment is zero, compare the learning increments of all types and choose the type with the largest learning increment. When there is only one job left of all types, compare the forgetting decrements and select a type with

Algorithm 3: GH2

Data: $l_j, f_{ij}, v_j^0, p_{j,x_j}, L_j, U_j$, job types $m, n = \sum_{j=1}^m n_j$
Result: Minimum makespan $g(v_1, \dots, v_m; x_1, \dots, x_m)$

```

1 Initialize  $P \leftarrow \emptyset; J \leftarrow \text{unscheduled jobs}$ ;
2 while  $J \neq \emptyset$  do
3   for each  $(j, x_j) \in J$  do
4     if  $x_j + 1 \leq n_j$  then
5        $\delta_l^+(j, x_j) \leftarrow p_{j,x_j+1} / v_j - p_{j,x_j} / v'_j$ , where  $v'_j = \min\{v_j + l_j p_{j,x_j}, U_j\}$ ;
6     end
7     else
8        $\delta_l^+(j, x_j) \leftarrow -1$ ;
9     end
10     $\delta_f^-(j, x_j) \leftarrow \sum_{h \neq j}^m (p_{h,x_h} / v'_h - p_{h,x_h} / v_h)$ , where  $v'_h = \max\{v_h - f_{jh} p_{j,x_j}, L_h\}$ ;
11  end
12   $S \leftarrow \text{None}$ ;
13  if  $\min_{(j,x_j) \in J} \{\delta_l^+(j, x_j)\} = -1$  then
14     $S \leftarrow \arg\min_{(j,x_j) \in J} \{\delta_f^-(j, x_j)\}$ ;
15  end
16  else
17    if  $\min_{(j,x_j) \in J} \{\delta_f^-(j, x_j)\} = 0$  then
18       $S \leftarrow \arg\max_{(j,x_j) \in J} \{\delta_l^+(j, x_j)\}$ ;
19    end
20    else
21       $S \leftarrow \arg\max_{(j,x_j) \in J} \{\delta_l^+(j, x_j) / \delta_f^-(j, x_j)\}$ ;
22    end
23  end
24  Process  $S$ ; Add  $S$  to  $P$ ; Remove  $S$  from  $J$ ; Update
     $g(v_1, \dots, v_m; x_1, \dots, x_m)$ ;
25 end
26 return  $g(v_1, \dots, v_m; x_1, \dots, x_m)$ ;

```

smallest forgetting decrement for processing. Finally, implement local search (see Algorithm 4) by swapping all adjacent pairs of jobs of different types, and output the resulting schedule with the smallest makespan. We also provide a numerical example of Algorithm GH4 in [Appendix A.3](#).

Algorithm 4: local search

Input : A feasible solution π
Output: An enhanced solution π

```

1 repeat
2    $is\_improved \leftarrow false$ ;
3   for  $j \in \pi$  do
4     if  $j$  is not last job,  $k$  is immediately after job  $j$  and, types
       of jobs  $k$  and  $j$  are not same then
5        $\pi' \leftarrow \text{swap jobs } j \text{ and } k$ ;
6       if  $f(\pi') < f(\pi)$  then
7          $\pi \leftarrow \pi'$ ;
8          $is\_improved \leftarrow true$ ;
9       end
10    end
11  end
12 until  $is\_improved = false$ ;
13 return  $\pi$ ;

```

Based on preliminary computational testing, we find that heuristics GH1-GH3 perform well for some instances, but inconsistently. However, GH4 demonstrates strong robust performance when used as a standalone heuristic and allows for easy implementation. Therefore, in our paper, we utilize heuristics GH1-GH3 as initial solutions to enhance

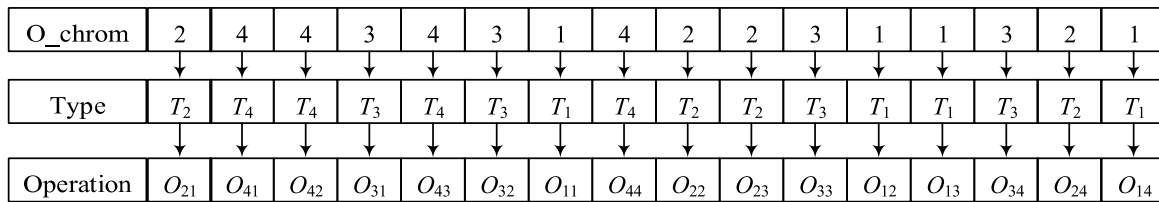


Fig. 2. Structure of operation chromosome.

genetic algorithm (GA) quality, while GH4 is used as a benchmark standalone heuristic for comparison with the GA.

5. Genetic algorithm

GAs are grounded in robust theoretical research and have widespread applications in engineering (Cheng et al., 1999; Qaraad et al., 2024). In this section, we present a hybrid adaptive GA to solve problem SLF, integrating problem-specific fitness calculations that incorporate learning and forgetting effects to achieve high-quality feasible solutions. The crossover process selects individuals with differing objectives to ensure genetic diversity, employing multiple crossover points for gene swapping. Additionally, the mutation process is adaptive, adjusting mutation rates based on individual fitness relative to the population average. This approach ensures a balance between exploration and exploitation, enhancing the algorithm's efficiency and effectiveness in solving problem SLF.

5.1. Encoding and decoding

To address the single-machine scheduling problem considering learning and forgetting effects, we employ a direct encoding method where each integer represents a specific job type. Fig. 2 provides an example of the operation chromosome structure. There are 16 jobs divided into four different types, represented by the numbers 1, 2, 3, and 4, which correspond to job types T_1 , T_2 , T_3 , and T_4 , respectively. The length of the operation chromosome is also the number of all operations. Operations are specified for each job, in accordance with O_{j,x_j} sequential processing, where x_j denotes the number of jobs for type j , $j \in \{1, 2, 3, 4\}$ (i.e., 4 jobs for each type).

5.2. Initialization

Previous research (Abreu et al., 2020; Wu et al., 2022) shows that the quality of the initial population significantly affects the convergence speed and search efficiency of the algorithm. Therefore, we design and test five initialization rules to ensure the quality and diversity of the initial population. The initial population is generated using the five methods of random selection, GH1, GH2, GH3, and choice of shortest processing time.

When implemented within the GA, GH1 prioritizes jobs by maximizing the learning effect while minimizing the forgetting effect, promoting a balanced approach that can improve overall performance. GH2 focuses on optimizing the immediate next step by selecting the job that offers the best trade-off between learning increment and forgetting increment. This localized optimization improves short-term scheduling decisions, cumulatively resulting in better overall performance. GH3 adopts a broader perspective by considering all remaining jobs for both learning increment and forgetting increments. This approach leads to more robust and effective scheduling decisions, benefiting the entire operation sequence. The SPT rule quickly realizes learning effects and reduces the impact of forgetting effects, thereby maintaining the efficiency of the schedule. These procedures increase the diversity and quality of the initial population by combining randomly generated solutions with those generated by heuristic methods.

5.3. Crossover and adaptive mutation probability

The performance of GAs also relies significantly on its crossover and mutation operators, which are crucial components of its global search capability. The values of the crossover probability P_c and mutation probability P_m are two key control parameters that affect the performance and convergence of the GA. In the crossover operation, individuals from the population are selected and paired to exchange genetic material. Managing the number of individuals and the genes to be crossed over is complex, since it must align with the changing fitness of individuals. In our implementation, we consider a fixed sequence within one job type and randomly select two distinct individuals from the population. For each gene in the selected individuals, if a random value is less than the crossover probability, the gene is marked for crossover. The marked genes are then swapped between the two individuals at the corresponding positions, generating new offspring and preserving genetic diversity within the population. This process is repeated until the desired number of crossover individuals is identified.

The mutation probability P_m models the exploitation vs. exploration tradeoff. A larger probability makes it easier to escape from a local extreme value to find the global optimal solution but a value of P_m that is too large reduces the algorithm to random search. On the other hand, if the P_m values are too small, it is difficult to produce new individuals, which makes evolution stagnate (Srinivas & Patnaik, 1994). As established in the GA literature (Liang et al., 2023), mutation probabilities P_m in the range $0.001 \leq P_m \leq 0.05$ are essential for the effective functioning of genetic algorithms. In our implementation, we use an adaptive mutation probability mechanism that addresses the constraints of a fixed sequence with any job type, where

$$P_m = \begin{cases} k_4, & \text{if } \gamma_i < \gamma_{avg}, \text{ where } k_4 \leq 1.0 \\ k_3 \frac{\gamma_{max} - \gamma_i}{\gamma_{max} - \gamma_{avg}}, & \text{if } \gamma_i \geq \gamma_{avg}, \text{ where } k_3 \leq 1.0. \end{cases}$$

First, the objective values of the individuals are calculated to determine the average (γ_{avg}) and maximum (γ_{max}) fitness values in the population. For individuals with fitness (γ_i) below the average, the mutation probability is set to a fixed value, k_4 . When the fitness of an individual is above the average, the mutation probability is adjusted linearly with respect to its fitness and the maximum fitness, using a value scaled by k_3 . Also, k_3 and k_4 have to be less than 1.0 to constrain P_m to the range 0.0–1.0. This process continues until the mutation population reaches the desired size. The adaptive mechanism allows poorly performing individuals to improve quickly while retaining some beneficial genetic material from better-performing individuals.

5.4. New population generation

The process of generating a new population involves combining the offspring generated by crossover and adaptive mutation with the original parent population to create an expanded pool of individuals. The quality of each individual in this combined pool is then evaluated using the value of its objective function. The objective function defined in our implementation calculates the makespan of each solution based on several factors, including processing times, learning values, and adjustments for learning and forgetting effects. Individuals in the

extended population are sorted in ascending order of their objective function values, reflecting the quality of each solution. This sorting process ensures that individuals with the lowest objective function values are prioritized. The selected N individuals, representing the best solutions from the combined pool, form the new population for the next iteration of the genetic algorithm. This updated population is used to continue the evolutionary process, focusing on further optimization and improvement of the solutions. The pseudocode for our implemented GA is as follows. Besides, a numerical example of Algorithm GA is presented in [Appendix A.4](#).

Algorithm 5: GA-New Population Generation

Input : Parameters **param** containing number of generations m_{gen} , population size n_{pop} , etc.
Output: Best individual in the population

```

1 Initialize population  $\mathbf{P} \leftarrow \text{INITIALIZEPOPULATION}(\mathbf{param})$ ;
2 for  $g \leftarrow 0$  to  $\mathbf{param}.m_{gen} - 1$  do
3    $\mathbf{C} \leftarrow \text{CROSSOVER}(\mathbf{P}, \mathbf{param})$ ;
4    $\mathbf{M} \leftarrow \text{MUTATION}(\mathbf{P}, \mathbf{param})$ ;
5    $\mathbf{A} \leftarrow \mathbf{P} \cup \mathbf{C} \cup \mathbf{M}$ ;
6   Sort  $\mathbf{A}$  by objective value in ascending order;
7    $\mathbf{P} \leftarrow$  first  $\mathbf{param}.n_{pop}$  individuals of  $\mathbf{A}$ ;
8 end
9 return the best individual from  $\mathbf{P}$ 

```

6. Lower bounds

We describe a procedure for finding a lower bound on optimal makespan for any instance of problem SLF. As a first step, we consider a special case of problem SLF without lower and upper bounds on learning, which we name problem SLC. This problem models a situation where the amount of learning for the different job types has closely converged as a result of previous processing, and therefore experiences only minor adjustments due to the current schedule. Such situations typically arise from extensive learning from high volume production. We provide a dynamic programming algorithm, SLC, for this problem.

Algorithm 6: SLC

Data: The learning effects l_j , the forgetting effects f_{hj} , the initial learning levels v_j^0 , and the nominal processing times of job p_{j,x_j} .
Result: The minimum makespan $g(x_1, \dots, x_m)$

```

1 Value Function:  $g(x_1, \dots, x_m)$  is the minimum makespan for processing jobs  $1, \dots, x_j$  of Type  $j$ , for  $j = 1, \dots, m$ ;
2 Boundary Condition:  $g(0, \dots, 0) = 0$ ;
3 Optimal Solution Value:  $g(n_1, \dots, n_m)$ ;
4 Recurrence Relations:

```

$$g(x_1, \dots, x_m) = \min_{1 \leq j \leq m} \{ \phi_j(v_j^0 + \sum_{k=1}^{x_j-1} l_j p_{j,k} - \sum_{h=1, h \neq j}^m \sum_{k=1}^{x_h} f_{hj} p_{h,k} x_j) + g(x_1, \dots, x_{j-1}, x_j - 1, x_{j+1}, \dots, x_m) \}.$$

Algorithm SLC begins with no jobs scheduled. The learning level of job j is initially v_j^0 , for $j = 1, \dots, m$. The recurrence relation repeatedly chooses a single job (j, x_j) , which is the x_j th job of Type j , to add to the schedule. This job has nominal processing time p_{j,x_j} . However, its actual processing time $\phi_j(v_j^0, x_j)$ depends on its current learning level v_j^0 , which is uniquely determined by the set of all previously scheduled jobs, independent of their processing sequence. Specifically, for each job Type j , the initial learning level v_j^0 is increased by a learning amount $\sum_{k=1}^{x_j-1} l_j p_{j,k}$ from the previous scheduling of jobs $(j, 1), \dots, (j, x_j - 1)$, and is reduced by a forgetting amount $\sum_{h=1, h \neq j}^m \sum_{k=1}^{x_h} f_{hj} p_{h,k}$ from the previous scheduling of jobs $(h, 1), \dots, (h, x_h)$, for $h = 1, \dots, j-1, j+1, \dots, m$. The algorithm terminates when all jobs have been scheduled.

Theorem 2. Algorithm SLC finds an optimal schedule for Problem SLC in $O(m \prod_{j=1}^m n_j)$ time.

Proof. The processing time of a job of any type is determined by its nominal processing time and the current learning level of that type. We observe that the state space $(x_1, \dots, x_{j-1}, x_j - 1, x_{j+1}, \dots, x_m)$ completely characterizes the information that is needed to compute the current learning level of job (j, x_j) and thereby also the remaining cost to complete the schedule. Hence, the recurrence relation compares the cost of all possible schedules, and finds an optimal schedule.

We note that $0 \leq x_j \leq n_j$, for $j = 1, \dots, m$. Further, the choice of job to schedule in the recurrence relation requires $O(m)$ comparisons. We calculate the values $\sum_{k=1}^{x_j-1} l_j p_{j,k}$ and $\sum_{h=1, h \neq j}^m \sum_{k=1}^{x_h} f_{hj} p_{h,k}$ incrementally in $O(m \prod_{j=1}^m n_j)$ time by a preprocessing procedure. Therefore, the overall time requirement of Algorithm SLC is $O(m \prod_{j=1}^m n_j)$ time. $\square$

We observe that the computational requirement of Algorithm SLC is much less than that of Algorithm SLF, since there is no need to store and update the learning levels of all the job types after each job is scheduled. This observation motivates a modification of Algorithm SLC that finds a lower bound for problem SLF, by adjusting for the effect of the lower bounds L_j and upper bounds U_j on the updated learning levels. We begin with the following preliminary result.

Lemma 1. (i). If all the learning levels computed by Algorithm SLC satisfy $L_j \leq v_j' \leq U_j$, $j = 1, \dots, m$, throughout the schedule, then those learning levels are accurate for problem SLF.

(ii). A learning level computed by Algorithm SLC that satisfies $v_j' < L_j$ results in subsequent learning levels being underestimated for problem SLF.

(iii). A learning level computed by Algorithm SLC that satisfies $v_j' > U_j$ results in subsequent learning levels being overestimated for problem SLF.

Proof. (i). Follows from the proof of [Theorem 2](#).

(ii). Follows from the correct learning level for problem SLF in Algorithm SLC being $\max\{v_j', L_j\}$.

(iii). Follows from the correct learning level for problem SLF in Algorithm SLC being $\min\{v_j', U_j\}$. $\square$

Next, we need the following preliminary result.

Lemma 2. The makespan of problem SLF with both learning and forgetting effects is a nonincreasing function of L_j, U_j , $j = 1, \dots, m$.

Proof. We consider a generic learning and forgetting effect. In the case of a learning effect for a job (i, j) , the learning level transition is $v_i' = v_i + \min\{l_i p_{i,j}, U_i - v_i\}$, which is clearly nondecreasing in U_i and independent of L_i . Further, the amount of forgetting of the other job types as a result of the processing of job (i, j) is independent of U_i and L_i . In the case of a forgetting effect for any job of Type $k \neq i$ as a result of the processing of job (i, j) , the learning level transition is $v_k' = v_k - \min\{f_{ik} p_{i,j}, v_k - L_k\}$, which is clearly nondecreasing in L_k and independent of U_k . Since the learning levels of all job types are nondecreasing in L_j, U_j , $j = 1, \dots, m$, it follows immediately that the makespan is nonincreasing in those parameters. $\square$

Since we need to find a lower bound for problem SLF, and from [Lemma 2](#) the duration of each job is a nonincreasing function of the learning level of its type, we may allow overestimation of learning levels under part (iii) of [Lemma 1](#). However, this overestimation should be kept to a minimum, in order to find a lower bound that is as tight as possible. On the other hand, underestimation of learning levels under part (ii) of [Lemma 1](#) can invalidate estimation of a lower bound for problem SLF. Hence, to ensure that a lower bound calculation is both valid and as strong as possible, it is necessary to maximize the possible underestimation in part (ii) and minimize the overestimation in part (iii) of all possible schedules leading to the current state configuration $(x_1, \dots, x_m)$. The following result identifies a restricted set of schedules that achieves this.

Lemma 3. Given any set of completed jobs, a processing sequence of those jobs where all the jobs of Type i , $1 \leq i \leq m$, $i \neq j$, precede all the jobs of Type j both (a) maximizes the underestimation of v'_j and (b) minimizes the overestimation of v'_j .

Proof. Given any set of completed jobs, the proof proceeds by pairwise interchange. Consider the first job of some Type $i \neq j$ that is processed immediately after a job of Type j . Interchanging these two jobs reduces the value of v'_j after the first job and leaves it unchanged after the second job. Repeating this interchange for all such pairs of jobs gives the minimum value of v'_j throughout the schedule, which achieves both properties (a) and (b) in the lemma statement. $\square$

The following observation defines the set of schedules that need to be computed to estimate the necessary adjustment to learning levels from the lower and upper bounds on learning.

Lemma 4. Properties (a) and (b) of Lemma 3 can both be achieved by scheduling any sequence of jobs of all job types except j , followed by scheduling all the jobs of Type j in their fixed sequence.

Proof. From Lemma 3, given that all the jobs of Type j are scheduled last, their possible overestimation is uniquely determined by the value of v'_j when the first of them starts processing their fixed sequence within Type j . Further, the value of v'_j when the first Type j job starts processing is $\max\{v_j^0 - \sum_{h=1, h \neq j}^m \sum_{k=1}^{x_h} f_{hj} p_{h,k}, L_j\}$, independent of the sequence of the jobs of other types, since all changes to v'_j from forgetting are reductions. $\square$

Based on Lemma 4 and the proof of Lemma 3, we describe an algorithm that achieves both properties (a) and (b). Given any set of completed jobs, this algorithm finds a valid upper bound on v'_j for any set of completed jobs. This value is then used to adjust the learning levels in Algorithm SLC. First, we describe our procedure to compute the necessary adjustments to the learning levels of each type, Algorithm SLF4.

Algorithm SLF4 computes an adjustment to the learning level of each job j , in two steps. The first step adjusts for saved forgetting when the scheduling of other job types results in a learning level of Type j that satisfies $v'_j < L_j$, in accordance with property (a) of Lemma 3. The second step adjusts for lost learning when the scheduling of Type j jobs results in a learning level of Type j that satisfies $v'_j > U_j$, in accordance with property (b) of Lemma 3. At the termination of Algorithm SLF4, the output is a table of learning level adjustments $\Delta_j(x_1, \dots, x_m)$, for $x_j = 0, \dots, n_j$, $j = 1, \dots, m$, that is used as a lookup table in the recurrence relations of Algorithm SLFLB which is described below.

Algorithm 7: SLF4

Data: The learning effects l_j , the forgetting effects f_{ij} , the initial learning levels v_j^0 , the nominal processing times of job p_{j,x_j} , the lower bounds on learning L_j , and the upper bounds on learning U_j .

Result: For any configuration $x_1, \dots, x_m$, where $0 \leq x_i \leq n_i$, $i = 1, \dots, m$, of scheduled jobs, an adjustment, $\Delta_j(x_1, \dots, x_m)$, to the learning level of Type j computed by Algorithm SLC.

```

1 For  $j = 1, \dots, m$  :
2   Reindex the job types such that Type  $j$  is Type 1.
3   For  $(x_2, \dots, x_m)$  :
4      $\Delta_j(x_1, \dots, x_m) = \max\{\sum_{i=2}^m \sum_{k=1}^{x_i} f_{ij} p_{i,k} - v_j^0 + L_j, 0\}$ .
5   End For
6    $\Delta_j(x_1, \dots, x_m) = \Delta_j(x_1, \dots, x_m) - \max\{\max\{L_j, v_j^0 - \sum_{i=2}^m \sum_{k=1}^{x_i} f_{ij} p_{i,k}\} + \sum_{k=1}^{x_j} l_j p_{j,k} - U_j, 0\}$ .
7 End For
```

Remark 1. We observe that, if the jobs other than those of Type j are all scheduled before any of Type j , then the upper bound on v_j is exact.

We now provide an example of the calculations in Algorithm SLF4.

Example 1. Data: $m = 3$; $n = 5$; $n_1 = 2, n_2 = 2, n_3 = 1$; $p_{1,1} = 300, p_{1,2} = 100, p_{2,1} = 100, p_{2,2} = 200, p_{3,1} = 300$; $L_j = 2, j = 1, 2, 3$; $U_1 = U_2 = 12, U_3 = 15$; $v_j^0 = 7, j = 1, 2, v_3^0 = 14$; $l_1 = .03, l_2 = .03, l_3 = .04$; $f_{12} = f_{13} = .01, f_{21} = .01, f_{23} = .02, f_{31} = .03, f_{32} = .02$. We show the calculations of $\Delta_j(2, 2, 1)$, $j = 1, 2, 3$, i.e. when all five jobs have been scheduled.

$$\begin{aligned} \Delta_1(2, 2, 1) &= \max\{1 + 2 + 9 - 7 + 2, 0\} - \max\{2 + 9 + 3 - 12, 0\} = 5. \\ \Delta_2(2, 2, 1) &= \max\{3 + 1 + 6 - 7 + 2, 0\} - \max\{2 + 3 + 6 - 12, 0\} = 5. \\ \Delta_3(2, 2, 1) &= \max\{3 + 1 + 2 + 4 - 14 + 2, 0\} - \max\{4 + 12 - 15, 0\} = -1. \end{aligned}$$

The following result demonstrates the usefulness of Algorithm SLF4.

Lemma 5. Algorithm SLF4 computes an adjustment to the learning levels in Algorithm SLC that, for any configuration of scheduled jobs, provides an upper bound on the correct learning level of Type j for $j = 1, \dots, m$, and does so in $O(\Pi_{j=1}^m n_j)$ time.

Proof. The validity of the upper bounds follows from Lemma 3 and Lemma 4.

We observe that $0 \leq x_h \leq n_h$, for $h = 1, \dots, m, h \neq j$. The summation terms in Algorithm SLF4 can be precomputed in $O(m^2 n)$ time and looked up as needed. Therefore, the overall computational requirement of Algorithm SLF4 is $O(\Pi_{j=1}^m n_j)$ time. $\square$

We now describe our overall procedure for finding a lower bound for Problem SLF, using Algorithm SLFLB below. This algorithm combines the state space structure of Algorithm SLC which is more computationally efficient than that of Algorithm SLF, with the necessary adjustments to learning levels described in Algorithm SLF4.

Algorithm 8: SLFLB

Data: The learning effects l_j , the forgetting effects f_{hj} , the initial learning levels v_j^0 , the nominal processing times of job p_{j,x_j} , the lower bounds on learning L_j , the upper bounds on learning U_j , and the adjustments $\Delta_j(x_1, \dots, x_m)$ found by Algorithm SLF4.

Result: A lower bound on the minimum makespan.

- Value Function:** $g(x_1, \dots, x_m)$ = the minimum makespan for processing jobs $1, \dots, x_j$ of Type j , for $j = 1, \dots, m$;
- Boundary Condition:** $g(0, \dots, 0) = 0$;
- Optimal Solution Value:** $g(n_1, \dots, n_m)$;

$$4 \text{ Recurrence Relations: } g(x_1, \dots, x_m) = \min_{1 \leq j \leq m} \left\{ \phi_j \left(v_j^0 + \sum_{k=1}^{x_j-1} l_j p_{j,k} - \sum_{h=1, h \neq j}^m \sum_{k=1}^{x_h} f_{hj} p_{h,k} + \Delta_j(x_1, \dots, x_{j-1}, x_j - 1, x_{j+1}, \dots, x_m), x_j \right) + g(x_1, \dots, x_{j-1}, x_j - 1, x_{j+1}, \dots, x_m) \right\}.$$

Algorithm SLFLB computes the minimum makespan sequence of jobs, similar to Algorithm SLC. However, at every scheduling decision, the learning level of each job type is an upper bound on the exact learning level that would be available at the same point in the schedule using Algorithm SLF. Since the processing time of each job is a nonincreasing function of its learning level, this property enables us to present the main result of this section.

Theorem 3. Algorithm SLFLB finds a valid lower bound for any instance of problem SLF in $O(m \Pi_{j=1}^m n_j)$ time.

Proof. Since, from Lemma 2, the processing time of any job is a decreasing function of the learning level of the job being scheduled, the maintenance of upper bounds on all learning levels provides a valid lower bound relaxation of problem SLF. As a result, the optimization of this relaxation by the comparison of all possible schedules in the

recurrence relation in Algorithm SLFLB finds a lower bound on optimal makespan.

The state variable v_j' in Algorithm SLFLB is computed directly from the set of previously scheduled jobs, and the relevant value of Δ is precomputed by Algorithm SLFA and accessed from a lookup table. Hence, the running time of the algorithm is the same as that of Algorithm SLC, as established in Theorem 2. $\square$

7. Computational study

The purpose of our computational study is to evaluate the performance of our solution procedures over a variety of data sets. We describe computational experiments using Heuristics GH1 through GH4, the GA, and Algorithm SLFDP to find feasible solutions, and Algorithm SLFLB to find lower bounds. Section 7.1 describes our experimental design. The effectiveness of the various optimal and heuristic solution procedures is estimated in Section 7.2. In Section 7.3, we perform a sensitivity analysis to study the effect of changes in various problem parameters. Section 7.4 addresses the effect of discretization of learning levels on our results.

7.1. Experimental design

All procedures are coded in C++. The experiments are performed on a computer with a CPU operating at 2.42 GHz with 288 GB of memory. The makespan objective is used to evaluate the solution accuracy, while the running time measures the efficiency of computation. Recall that there are m types of jobs, and we need to schedule n_j of each type j non-preemptively, resulting in a total of $n = \sum_{j=1}^m n_j$ jobs. We set the actual processing times of the jobs as in (1). Consistent with Yang and Chand (2008), the learning rates l_j are randomly generated from the range [0.61, 0.90] using a uniform distribution, and are then rounded to three decimal places. The parameters α_{1j} and α_{2j} are randomly generated from two ranges [0.900, 1.000] and [−0.100, 0.000] respectively, using a uniform distribution, and are then rounded to three decimal places. Following Pan et al. (2014), the similarity r_{ij} between the type of the job being processed and the type of the job waiting for processing is modeled within the range [0.95, 0.99], resulting in $f_{ij} = 1 - r_{ij}$, and thus $f_{ij} \in [0.01, 0.05]$ using a uniform distribution, and then rounded to three decimal places, which ensures $f_{ij} \leq l_j$. Without loss of generality, processing times are randomly generated integers from the uniform distribution defined on [1, 40] (Yang & Chand, 2008). For the GA, the number of iterations is set to 200, and the population size is set to 100. The crossover rate is set at $P_c = 0.7$, with mutation rates $k_3 = 0.1$ and $k_4 = 0.2$.

7.2. Solution quality

To evaluate the performance of the Algorithms SLFDP and SLFLB, the GA, and Heuristic GH4, we test instances with several values of n and m . We first consider small instances with 7, 9, 11, 13, and 15 jobs, and 2, 4, 6, and 8 job types. Each instance is solved 10 times to obtain the mean result, and the results are presented in Table 1. The table provides information on the execution time in seconds to obtain the optimal solution using SLFDP. For each combination of n and m , the following performance metrics are calculated: (a) the ratio of the average error of Algorithm SLFLB with respect to the optimal solution, denoted as $Gap_{LB}(\%) = 100 \times \frac{DP-LB}{DP}$; (b) the ratio of the average error of the GA with respect to the optimal solution, denoted as $Gap_{GA}(\%) = 100 \times \frac{GA-DP}{DP}$; (c) the ratio of the average error of the Heuristic GH4 with respect to the optimal solution, denoted as $Gap_H(\%) = 100 \times \frac{H-DP}{DP}$; and (d) the execution time in seconds. From the table, it can be seen that SLFLB, GA and GH4 are able to obtain near-optimal solutions for all instances within a few seconds. Moreover, for all small instances, the average objective value of the optimality gap for GA is zero, demonstrating the effectiveness of the GA in finding the

optimal solution for all 10 instances run. The results also show that as the number of jobs increases, it becomes harder for Algorithm SLFDP to solve to optimality within a reasonable amount of computation time. For example, this is evident in the 6×15 and 8×15 tests, where some instances require longer than the 1,000 s limit imposed.

For larger instances, Algorithm SLFDP cannot be used to evaluate the performance of the GA and GH4 due to its computational complexity. To check the efficiency and quality of the solutions obtained by the GA and GH4, we compare upper bound solutions found by the GH4 and the GA with lower bounds found by Algorithm SLFLB, and report the percentage gap between them. For this purpose, we test instances with varying n and m , including 30, 40, 50, 60 and 80 jobs and 2, 4, 6 and 8 job types. Each combination of n and m is tested over 10 instances, and the mean results are presented in Table 2. For each combination of n and m , the values of $Gap_{GA}(\%)$ by the GA, which represent the gaps between the GA solutions and the SLFLB lower bounds, are much smaller than the values of $Gap_H(\%)$, which represent the gaps between the GH4 solutions and the SLFLB lower bounds. The gaps for the GA are typically well below 1%, indicating the effectiveness of this procedure for solving large instances of problem SLF. Additionally, it can be seen that both GA and GH4 are able to obtain near-optimal solutions for all instances within a few seconds.

7.3. Sensitivity analysis

In this section, we perform a sensitivity analysis by changing the value of certain parameters of interest, such as α_{1j} and α_{2j} for calculating $\phi_j(v_j', x_j)$, as well as U_j and L_j . For a clear comparison, we set the sizes of $m \times n$ to 4×7 , 4×9 , 4×20 and 4×40 . The learning rate is set within [0.61, 0.90] and the forgetting rate is set as $f_{ij} \in [0.01, 0.05]$, both following a uniform distribution and rounded to three decimal places.

For α_{1j} , we examine the problem from a gap measurement perspective, with α_{2j} fixed in the range [−0.100, 0.000]. Table 3 shows the variation in the values of SLFLB, GA and GH4 across different combinations of n and m , as the values of α_{1j} decrease. For small instances where $n \in \{7, 9\}$ and $[L_j, U_j] = [2, 40]$, the value of $Gap_{LB}(\%)$ initially increases and then decreases, reaching a high level when $n = 7$, $\alpha_{1j} \in [−0.200, 0.300]$, as well as $n = 9$, $\alpha_{1j} \in [−0.600, 0.700]$. The value of $Gap_{GA}(\%)$ remains consistently zero, while $Gap_H(\%)$ reaches a high level when $n = 7$, $\alpha_{1j} \in [0.100, 0.200]$, and again when $\alpha_{1j} \in [−0.600, 0.700]$ and $n = 9$. For large instances where $n \in \{20, 40\}$ and $[L_j, U_j] = [2, 60]$, the value of $Gap_{GA}(\%)$ follows a similar trend of initially increasing and then decreasing, whereas the value of $Gap_H(\%)$ decreases consistently. As α_{1j} initially decreases, the influence of learning and forgetting effects decreases and the actual processing time approaches the nominal processing time. Here, due to the lack of significant information from the learning and forgetting effects, the GA and heuristic algorithms may find it more difficult to reach the optimal solution, leading to an increase in the gap value. As α_{1j} continues to decrease and approaches 0, the processing time is almost unaffected by learning and forgetting effects and becomes relatively fixed. In this scenario, the scheduling problem becomes much simpler and the accuracy of the GA and heuristic algorithms improves, thereby reducing the gap again.

For α_{2j} , we examine the problem from a gap measurement perspective, with α_{1j} fixed in the range [0.900, 1.000]. Table 4 shows the variation in the values of SLFLB, GA and GH across different combinations of n and m , as the values of α_{2j} decrease. For small instances, where $[L_j, U_j] = [2, 40]$, the value of $Gap_{LB}(\%)$ decreases, and the value of $Gap_{GA}(\%)$ remains consistently zero, while $Gap_H(\%)$ shows an increasing trend. For large instances, with $[L_j, U_j] = [2, 60]$, the value of $Gap_{GA}(\%)$ increases when $n = 20$ and follows a trend of initially increasing and then decreasing when $n = 40$, while $Gap_H(\%)$ increases consistently. In (1), the term $(\sum_{i=1}^m x_i)^{\alpha_{2j}}$ represents the position of the current job in the schedule. A smaller α_{2j} value implies that as the number of tasks increases, the impact of the learning effect on reducing

Table 1
Results for Small Instances.

m	n	$[L_j, U_j]$	$SLFDP$	$SLFLB$		GA		$GH4$	
			Time	$Gap_{LB}(\%)$	Time	$Gap_{GA}(\%)$	Time	$Gap_H(\%)$	Time
2	7	[2, 20]	0.012	1.377	3.87e-05	0.000	0.059	0.000	1.35e-05
	9		0.008	0.325	5.57e-05	0.000	0.079	0.000	1.77e-05
	11		0.010	0.687	7.92e-05	0.000	0.085	0.000	3.22e-05
	13		0.011	0.254	0.000	0.000	0.085	0.000	2.87e-05
	15		0.042	1.107	0.000	0.000	0.09	0.424	3.54e-05
4	7	[2, 40]	0.007	0.042	0.000	0.000	0.06	0.000	1.06e-05
	9		0.022	0.000	0.000	0.000	0.062	0.000	1.14e-05
	11		0.165	0.059	0.000	0.000	0.068	0.000	2.80e-05
	13		2.265	0.651	0.000	0.000	0.109	1.164	3.44e-05
	15		23.148	0.000	0.001	0.000	0.087	4.24	5.57e-05
6	7	[2, 60]	0.018	0.000	0.000	0.000	0.058	1.637	1.25e-05
	9		0.143	1.88e-14	0.000	0.000	0.068	0.000	3.01e-05
	11		1.995	0.561	0.001	0.000	0.066	0.141	3.46e-05
	13		56.142	0.000	0.002	0.000	0.078	0.947	1.78e-05
	15		--	--	0.006	0.032	0.081	2.515	3.62e-05
8	9	[2, 80]	0.868	0.047	0.001	0.000	0.066	0.000	3.07e-05
	11		17.775	0.031	0.003	0.000	0.101	1.143	4.57e-05
	13		952.626	0.310	0.0138	0.000	0.108	1.809	7.70e-05
	15		--	--	0.019	0.070	0.108	3.349	5.24e-05

Table 2
Results for large instances.

m	n	$[L_j, U_j]$	$SLFLB$	GA		$GH4$	
			Time	$Gap_{GA}(\%)$	Time	$Gap_H(\%)$	Time
2	30	[2, 40]	0.000	0.000	0.119	0.257	0.000
	40		0.000	0.000	0.140	0.563	0.000
	50		0.001	0.000	0.163	0.696	0.000
	60		0.002	0.000	0.222	0.688	0.000
	80		0.003	0.000	0.301	0.479	0.000
4	30	[2, 60]	0.020	0.006	0.140	3.920	0.000
	40		0.063	0.009	0.175	2.131	0.000
	50		0.126	0.079	0.197	3.005	0.000
	60		0.281	0.087	0.220	3.651	0.001
	80		0.859	0.073	0.297	3.222	0.003
6	30	[2, 80]	0.176	0.024	0.153	3.372	0.000
	40		0.953	0.056	0.175	4.032	0.000
	50		3.427	0.070	0.240	4.121	0.001
	60		10.107	0.061	0.238	5.744	0.002
	80		69.917	0.098	0.357	4.751	0.002
8	30	[2, 100]	1.133	0.016	0.149	5.608	0.000
	40		11.981	0.027	0.171	3.133	0.000
	50		48.521	0.035	0.314	6.719	0.001
	60		179.135	0.060	0.278	1.665	0.004
	80		2239.31	0.023	0.351	7.003	0.006

the processing time becomes less significant and the processing times of each task become more uniform. In this case, it becomes more difficult for heuristic algorithms to establish a clear priority order when selecting the task sequence, leading to an increased gap value between the GH4 solution and the exact solution. However, a sufficient reduction in α_{2j} makes the position of the job less relevant, which leads to a decrease in the gap value.

We consider $[L_j, U_j]$ from a gap measurement perspective, with $\alpha_{1j} \in [0.900, 1.000]$ and $\alpha_{2j} \in [-0.100, 0.000]$, as shown in Table 5. For small instances, the value of $Gap_{LB}(\%)$ increases initially and then stabilizes when $n = 7$, reaching a high value when $n = 9$ and $[L_j, U_j] = [2, 20]$, and thereafter is zero. The value of $Gap_{GA}(\%)$ and $Gap_H(\%)$ remains consistently zero. For large instances, the values of $Gap_{GA}(\%)$ and $Gap_H(\%)$ follow a trend of initially decreasing and then increasing. When the interval between U_j and L_j is small, the relative performance of GA and GH4 weakens due to the lower bound approximation in Algorithm SLF4, resulting in a larger gap value. As the interval between U_j and L_j increases, GA and GH4 can make better use of their optimization capabilities, resulting in a gradual decrease in the gap value. However, as the interval between U_j and L_j continues

to increase, GA and GH4 may find local rather than global optima in the larger solution space, hence the gap value increases again but eventually stabilizes at about 0.3% and 2.5%, respectively.

7.4. Discretization level

Our computational results, as reported in Sections 7.2 and 7.3, utilize integer nominal processing times and l_j and f_{ij} values with no more than three decimal places. Consequently, the discretization of the learning levels v'_j also employs a precision of three decimal places. To ensure that this level of discretization does not introduce errors in estimating the performance of our solution procedures, we conduct additional computational tests, as shown in Table 6. These tests are carried out with $m = 8$ and $n = 30, 40, 50, 60, 80$, using three decimal places for both l_j and f_{ij} . The results indicate that there are no significant differences compared to the tests reported in Section 7.2, which confirms the robustness and reliability of our discretization approach.

8. Concluding remarks

This paper studies a production scheduling problem that arises in human-machine manufacturing environments. The processing time of a job is determined by a learning level based on machine adjustments, and on the number of previously scheduled jobs which models human learning. Jobs are classified into types. When a job of a particular type is processed, the learning level of that type increases, but the learning level of all other types decreases due to forgetting. However, the learning level of a job type has both upper and lower bounds. The objective studied is minimization of the makespan. To find feasible solutions, we develop greedy-type heuristics and a hybrid adaptive genetic algorithm. The quality of the solutions found is validated by lower bounds obtained from a dynamic programming algorithm that modifies an optimal algorithm which we describe for the problem without upper and lower limits on learning levels. These procedures combine to deliver routinely close to optimal solutions for makespan minimization for instances with up to eight types and 80 jobs. Our results also validate simple scheduling rules that routinely provide close to optimal solutions with minimal implementation and computation time. Detailed computational experiments identify the sensitivity of performance of our solution approaches to various parameter changes, which yields insights about the tradeoffs in our model.

Our work provides opportunities for operations managers that previous work on scheduling with learning and forgetting do not. The exact

Table 3
Sensitivity analysis for α_{1j} .

$\alpha_{2j} \in [-0.100, 0.000] \ \& [L_j, U_j] = [2, 40]$						$\alpha_{2j} \in [-0.100, 0.000] \ \& [L_j, U_j] = [2, 60]$				
m	n	α_{1j}	$Gap_{LB}(\%)$	$Gap_{GA}(\%)$	$Gap_H(\%)$	m	n	α_{1j}	$Gap_{GA}(\%)$	$Gap_H(\%)$
4	7	[0.900,1.000)	0.042	0.000	0.000	4	20	[0.900,1.000)	0.527	5.005
		[0.800,0.900)	0.045	0.000	0.000			[0.800,0.900)	0.530	4.865
		[0.700,0.800)	0.048	0.000	0.000			[0.700,0.800)	0.548	4.668
		[0.600,0.700)	0.049	0.000	0.000			[0.600,0.700)	0.580	4.402
		[0.500,0.600)	0.049	0.000	0.000			[0.500,0.600)	0.623	4.061
		[0.400,0.500)	0.048	0.000	0.000			[0.400,0.500)	0.453	3.635
		[0.300,0.400)	0.044	0.000	0.000			[0.300,0.400)	0.169	3.120
	9	[0.200,0.300)	0.087	0.000	0.000		40	[0.200,0.300)	0.127	2.690
		[0.100,0.200)	0.086	0.000	0.048			[0.100,0.200)	0.111	2.065
		[0.900,1.000)	0.125	0.000	0.000			[0.900,1.000)	0.233	2.130
		[0.800,0.900)	0.132	0.000	0.000			[0.800,0.900)	0.286	2.165
		[0.700,0.800)	0.308	0.000	0.000			[0.700,0.800)	0.193	2.148
		[0.600,0.700)	0.312	0.000	0.000			[0.600,0.700)	0.045	2.363
		[0.500,0.600)	0.307	0.000	0.000			[0.500,0.600)	0.047	2.558
		[0.400,0.500)	0.29	0.000	0.000			[0.400,0.500)	0.050	2.535
		[0.300,0.400)	0.264	0.000	0.000			[0.300,0.400)	0.000	2.419
		[0.200,0.300)	0.271	0.003	0.007			[0.200,0.300)	0.000	2.128
		[0.100,0.200)	0.186	0.000	0.275			[0.100,0.200)	0.003	1.363

Table 4
Sensitivity Analysis for α_{2j} .

$\alpha_{1j} \in [0.900, 1.000] \ \& [L_j, U_j] = [2, 40]$						$\alpha_{1j} \in [0.900, 1.000] \ \& [L_j, U_j] = [2, 60]$				
m	n	α_{2j}	$Gap_{LB}(\%)$	$Gap_{GA}(\%)$	$Gap_H(\%)$	m	n	α_{2j}	$Gap_{GA}(\%)$	$Gap_H(\%)$
4	7	[-0.100, 0.000)	0.042	0.000	0.000	4	20	[-0.100, 0.000)	0.527	5.005
		[-0.200, -0.100)	0.042	0.000	0.000			[-0.200, -0.100)	0.493	5.571
		[-0.300, -0.200)	0.041	0.000	0.000			[-0.300, -0.200)	0.000	9.780
		[-0.400, -0.300)	0.040	0.000	0.000			[-0.400, -0.300)	0.000	15.840
		[-0.500, -0.400)	0.000	0.000	1.208			[-0.500, -0.400)	0.000	21.805
		[-0.600, -0.500)	0.000	0.000	3.324			[-0.600, -0.500)	0.000	27.942
		[-0.700, -0.600)	0.000	0.000	5.379			[-0.700, -0.600)	0.000	33.177
	9	[-0.800, -0.700)	0.000	0.000	7.353		40	[-0.800, -0.700)	0.000	37.840
		[-0.900, -0.800)	0.000	0.000	9.230			[-0.900, -0.800)	0.000	41.818
		[-0.100, 0.000)	0.000	0.000	0.000			[-0.100, 0.000)	0.233	2.130
		[-0.200, -0.100)	0.099	0.000	0.000			[-0.200, -0.100)	0.700	5.642
		[-0.300, -0.200)	0.187	0.000	0.019			[-0.300, -0.200)	3.721	10.898
		[-0.400, -0.300)	0.178	0.000	0.128			[-0.400, -0.300)	1.089	16.234
		[-0.500, -0.400)	0.168	0.000	0.000			[-0.500, -0.400)	0.000	23.178
		[-0.600, -0.500)	0.158	0.000	0.000			[-0.600, -0.500)	0.000	32.407
		[-0.700, -0.600)	0.149	0.000	0.000			[-0.700, -0.600)	0.000	42.602
		[-0.800, -0.700)	0.139	0.000	0.043			[-0.800, -0.700)	0.000	51.483
		[-0.900, -0.800)	0.000	0.000	0.000			[-0.900, -0.800)	0.000	60.587

Table 5
Sensitivity Analysis for L_j and U_j .

m	n	$[L_j, U_j]$	$Gap_{LB}(\%)$	$Gap_{GA}(\%)$	$Gap_H(\%)$	m	n	$[L_j, U_j]$	$Gap_{GA}(\%)$	$Gap_H(\%)$
4	7	[2, 20]	0.042	0.000	0.000	4	20	[2, 20]	0.606	5.585
		[2, 40]	0.042	0.000	0.000			[2, 40]	0.541	4.651
		[2, 60]	0.043	0.000	0.000			[2, 60]	0.527	5.005
		[2, 80]	0.043	0.000	0.000			[2, 80]	0.528	5.213
		[2, 100]	0.043	0.000	0.000			[2, 100]	0.531	5.321
		[2, 120]	0.043	0.000	0.000			[2, 120]	0.531	5.321
		[2, 140]	0.043	0.000	0.000			[2, 140]	0.531	5.321
	9	[2, 160]	0.043	0.000	0.000		40	[2, 160]	0.531	5.321
		[2, 20]	0.125	0.000	0.000			[2, 20]	0.445	3.357
		[2, 40]	0.000	0.000	0.000			[2, 40]	0.184	2.088
		[2, 60]	0.000	0.000	0.000			[2, 60]	0.233	2.130
		[2, 80]	0.000	0.000	0.000			[2, 80]	0.281	2.277
		[2, 100]	0.000	0.000	0.000			[2, 100]	0.295	2.392
		[2, 120]	0.000	0.000	0.000			[2, 120]	0.309	2.489
		[2, 140]	0.000	0.000	0.000			[2, 140]	0.311	2.497
		[2, 160]	0.000	0.000	0.000			[2, 160]	0.320	2.537

positioning of our work relative to the previous literature is illustrated in Fig. 1. Specifically, we believe that our work is the first to enable the optimal or near-optimal solution of large scheduling problems with learning and full matrix similarity forgetting. Our modeling of actual processing times is very general and can be modified using several parameters to match particular applications. For managers who prefer a fast and simple solution process, Greedy Heuristic GH4 provides solutions that are routinely within a few percentage points from optimal.

For more accurate solutions, the genetic algorithm provides solutions that are routinely within 1% of optimal. Further, their accuracy can be validated using Algorithm SLFLB. Our sensitivity analysis results identify characteristics of the problem studied that affect solvability. We briefly discuss limitations of our work. First, problem instances with much more than eight job types or 80 jobs may be difficult to solve with the same degree of accuracy. Second, in some applications, it may

Table 6
Discretization Level.

m	n	$[L_j, U_j]$	SLFLB	GA		GH4	
			Time	Gap _{GA} (%)	Time	Gap _H (%)	Time
8	30	[2,100]	1.032	0.073	0.145	6.928	0.000
	40		9.869	1.249	0.206	2.228	0.000
	50		62.348	0.189	0.227	4.211	0.000
	60		225.301	0.598	0.288	4.319	0.000
	80		2119.685	2.557	0.417	6.052	0.001

be difficult to obtain accurate estimates of the learning and forgetting coefficients.

Several valuable directions remain open for future research. First, while our modeling of actual job processing time based on learning level and job position is consistent with the human-machine literature, there are various alternative designs that can be employed. Indeed, the methodology we describe can be used to explore these designs. Second, we provide a detailed discretization of learning levels, but even more detailed discretizations can be used without increasing the computational difficulty of the problem. Third, while we consider a single machine environment, it is possible to extend our work to study a parallel machine environment or other more general manufacturing configurations. Fourth, a natural generalization of the problem studied here would allow for multiple batches of jobs, for example due to a capacity limitation on a single batch. In that situation, the nominal and initial times of jobs may be different for the second and later batches, however all our results remain valid in that more general setting. Fifth, the effect of learning and forgetting on the quality of work should be studied in human-machine workplaces. Sixth, r_{ij} is currently determined randomly; however, it can be refined using more structured approaches, such as the method proposed by Jaber et al. (2003) for defining a similarity coefficient between tasks. Finally, while our work considers the objective of minimizing makespan, it should be possible to consider other objectives such as the minimization of the weighted number of late jobs relative to given due dates.

CRediT authorship contribution statement

Shuling Xu: Writing – original draft, Methodology, Funding acquisition, Data curation, Conceptualization. **Fulong Xie:** Writing – original draft, Validation, Software, Data curation. **Nicholas G. Hall:** Visualization, Validation, Supervision, Project administration, Methodology, Formal analysis, Conceptualization.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant nos. 72201112, 72271070, 71871076); and by the Youth Program of Humanities and Social Sciences of the Ministry of Education of China (Grant no. 22YJCZH205).

Appendix

A.1. Learning-forgetting curves

In our work, the amount of learning and forgetting depends on the full matrix similarity of the two job types, and on the size of the jobs processed. Fig. 3 illustrates the learning-forgetting process performing a sequence of eight jobs $(p_{(1,1)}, p_{(2,1)}, p_{(3,1)}, p_{(1,2)}, p_{(1,3)}, p_{(3,2)}, p_{(3,3)}, p_{(3,4)})$ with three different types, i.e. $T_j, j \in \{1, 2, 3\}$, and (a), (b) and (c) show the SLF patterns of the jobs of T_j . For $\forall j, j \in \{1, 2, 3\}$, v_j^i indicates the current learning level of jobs for type T_j ; v_j^0 denotes the initial learning level of jobs for type T_j ; U_j and L_j represent the upper limit and the lower limit of the learning level of jobs for type T_j respectively.

A.2. Numerical example of Algorithm SLFDP

Example 2. Data: $m = 3; n = 5; n_1 = 2, n_2 = 2, n_3 = 1 : p_{1,1} = 300, p_{1,2} = 100, p_{2,1} = 100, p_{2,2} = 200, p_{3,1} = 300; L_j = 2, U_j = 12, v_j^0 = 7, j = 1, 2, 3; l_1 = .02, l_2 = .01, l_3 = .01; f_{12} = f_{13} = .01, f_{21} = .01, f_{23} = .02, f_{31} = .03, f_{32} = .02$. Our formulation of problem SLF is quite general, and allows for many different models to calculate $\phi_j(\cdot, \cdot)$. However, to simplify the following example, we generalize the learning function of Wright (1936) and use $\phi_j(v_j^i, x_j) = p_{j,x_j}/v_j^i$, for $i = 1, 2, 3$, and $j = 1, \dots, n_i$. In the solution that follows, the highlighted states are those that are identified by backtracking as being on an optimal path at the end of the algorithm.

Solution

Boundary condition:

$g(7, 7, 7; 0, 0, 0) = 0$.

First job choice:

$g(12, 4, 4; 1, 0, 0) = 300/7 + g(7, 7, 7; 0, 0, 0) = 42.857$

$g(6, 8, 5; 0, 1, 0) = 100/7 + g(7, 7, 7; 0, 0, 0) = 14.286$

$g(2, 2, 10; 0, 0, 1) = 300/7 + g(7, 7, 7; 0, 0, 0) = 42.857$.

Second job choice:

$g(12, 2, 2; 2, 0, 0) = 100/12 + g(12, 4, 4; 1, 0, 0) = 51.190$

$g(11, 5, 2; 1, 1, 0) = 100/4 + g(12, 4, 4; 1, 0, 0) = 67.857$

$g(12, 5, 2; 1, 1, 0) = 300/6 + g(6, 8, 5; 0, 1, 0) = 64.286$

$g(3, 2, 7; 1, 0, 1) = 300/4 + g(12, 4, 4; 1, 0, 0) = 117.857$

$g(8, 2, 7; 1, 0, 1) = 300/2 + g(2, 2, 10; 0, 0, 1) = 192.857$

$g(2, 3, 8; 0, 1, 1) = 100/2 + g(2, 2, 10; 0, 0, 1) = 92.857$

$g(2, 2, 8; 0, 1, 1) = 300/5 + g(6, 8, 5; 0, 1, 0) = 74.286$

$g(4, 10, 2; 0, 2, 0) = 200/6 + g(6, 8, 5; 0, 1, 0) = 76.190$.

Third job choice:

$g(12, 4, 2; 2, 1, 0) = 100/12 + g(12, 5, 2; 1, 1, 0) = 72.619$

$g(11, 5, 2; 2, 1, 0) = 100/2 + g(12, 2, 2; 2, 0, 0) = 101.190$

$g(5, 2, 6; 2, 0, 1) = 100/3 + g(3, 2, 7; 1, 0, 1) = 151.190$

$g(10, 2, 6; 2, 0, 1) = 100/5 + g(8, 2, 7; 1, 0, 1) = 212.857$

$g(3, 2, 5; 2, 0, 1) = 300/2 + g(12, 2, 2; 2, 0, 0) = 201.190$

$g(9, 7, 2; 1, 2, 0) = 200/5 + g(11, 5, 2; 1, 1, 0) = 107.857$

$g(10, 7, 2; 1, 2, 0) = 200/5 + g(12, 5, 2; 1, 1, 0) = 104.286$

$g(10, 9, 2; 1, 2, 0) = 300/4 + g(4, 10, 2; 0, 2, 0) = 151.190$

$g(2, 2, 5; 1, 1, 1) = 300/2 + g(11, 5, 2; 1, 1, 0) = 217.857$

$g(3, 2, 5; 1, 1, 1) = 300/2 + g(12, 5, 2; 1, 1, 0) = 214.286$

$g(2, 3, 5; 1, 1, 1) = 100/2 + g(3, 2, 7; 1, 0, 1) = 167.857$

$g(7, 3, 5; 1, 1, 1) = 100/2 + g(8, 2, 7; 1, 0, 1) = 242.857$

$g(8, 2, 5; 1, 1, 1) = 300/2 + g(2, 2, 8; 1, 0, 1) = 224.286$

$g(2, 5, 4; 0, 2, 1) = 200/2 + g(2, 3, 8; 0, 1, 1) = 192.857$

$g(2, 4, 8; 0, 2, 1) = 200/2 + g(2, 2, 12; 0, 1, 1) = 174.286$

$g(2, 4, 5; 0, 2, 1) = 300/2 + g(4, 10, 2; 0, 2, 0) = 226.190$.

Fourth job choice:

$g(10, 6, 2; 2, 2, 0) = 200/4 + g(12, 4, 2; 2, 1, 0) = 126.948$

$g(9, 6, 2; 2, 2, 0) = 200/4 + g(12, 4, 2; 2, 1, 0) = 122.619$

$g(10, 7, 2; 2, 2, 0) = 200/5 + g(11, 5, 2; 2, 1, 0) = 141.190$

$g(10, 6, 2; 2, 2, 0) = 100/9 + g(9, 7, 2; 1, 2, 0) = 118.968$

$g(11, 6, 2; 2, 2, 0) = 100/10 + g(10, 7, 2; 1, 2, 0) = 114.286$

$g(11, 8, 2; 2, 2, 0) = 100/10 + g(10, 9, 2; 1, 2, 0) = 161.190$

$g(5, 2, 4; 2, 1, 1) = 100/3 + g(3, 2, 5; 1, 1, 1) = 247.619$

$g(4, 2, 4; 2, 1, 1) = 100/2 + g(2, 3, 5; 1, 1, 1) = 217.857$

$g(9, 2, 2; 2, 1, 1) = 100/7 + g(7, 3, 5; 1, 1, 1) = 307.143$

$g(10, 2, 4; 2, 1, 1) = 100/8 + g(8, 2, 5; 1, 1, 1) = 255.357$

$g(10, 2, 8; 2, 1, 1) = 100/9 + g(8, 2, 9; 1, 1, 1) = 235.397$

$g(4, 2, 4; 2, 1, 1) = 100/2 + g(5, 2, 6; 2, 0, 1) = 201.190$

$g(8, 3, 4; 2, 1, 1) = 100/2 + g(10, 2, 6; 2, 0, 1) = 262.857$

$g(2, 3, 3; 2, 1, 1) = 100/3 + g(3, 2, 5; 2, 0, 1) = 234.523$

$g(6, 3, 2; 2, 1, 1) = 100/2 + g(10, 2, 6; 2, 0, 1) = 262.857$

$g(3, 2, 5; 2, 1, 1) = 300/2 + g(12, 4, 2; 2, 1, 0) = 222.619$

$g(2, 2, 5; 2, 1, 1) = 300/2 + g(12, 4, 2; 2, 1, 0) = 222.619$.

$g(4, 3, 2; 1, 2, 1) = 300/2 + g(2, 4, 4; 0, 2, 1) = 342.857$

$g(8, 3, 2; 1, 2, 1) = 300/2 + g(2, 4, 8; 0, 2, 1) = 324.286$

$g(4, 3, 4; 1, 2, 1) = 300/2 + g(2, 4, 5; 0, 2, 1) = 376.190$

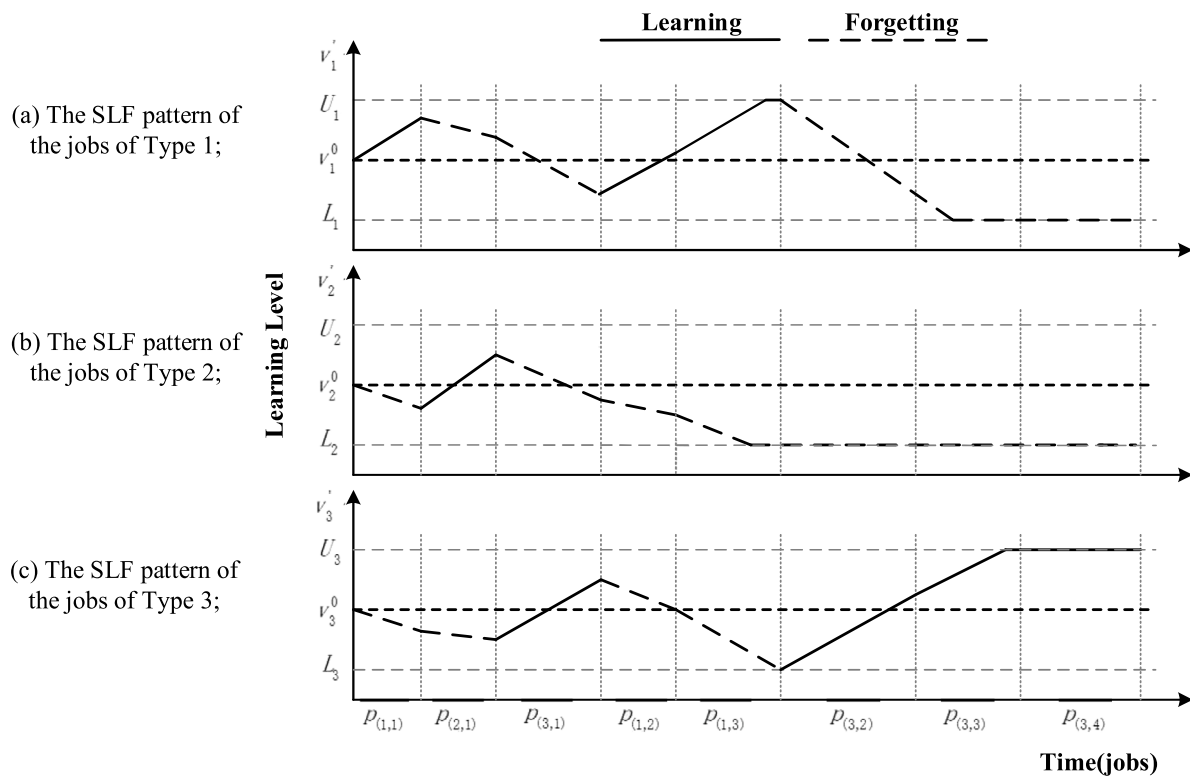


Fig. 3. Schematic illustration of cumulative learning and forgetting curves.

$g(2, 4, 2; 1, 2, 1) = 200/2 + g(3, 2, 5; 1, 1, 1) = 314.286$
 $g(2, 5, 2; 1, 2, 1) = 200/3 + g(2, 3, 5; 1, 1, 1) = 234.524$
 $g(5, 5, 2; 1, 2, 1) = 200/3 + g(7, 3, 5; 1, 1, 1) = 309.524$
 $g(6, 4, 2; 1, 2, 1) = 200/2 + g(8, 2, 5; 1, 1, 1) = 342.857$
 $g(6, 4, 5; 1, 2, 1) = 200/2 + g(8, 2, 9; 1, 1, 1) = 324.286$
 $g(2, 2, 6; 1, 2, 1) = 300/2 + g(9, 7, 2; 1, 2, 0) = 257.857$
 $g(2, 2, 5; 1, 2, 1) = 300/2 + g(10, 7, 2; 1, 2, 0) = 254.286$
 $g(2, 3, 5; 1, 2, 1) = 300/2 + g(10, 9, 2; 1, 2, 0) = 301.190$

Fifth job choice:

$g(6, 2, 2; 2, 2, 1) = 100/4 + g(4, 3, 2; 1, 2, 1) = 367.857$
 $g(10, 2, 2; 2, 2, 1) = 100/8 + g(8, 3, 2; 1, 2, 1) = 336.786$
 $g(6, 2, 3; 2, 2, 1) = 100/4 + g(4, 3, 4; 1, 2, 1) = 401.190$
 $g(4, 3, 2; 2, 2, 1) = 100/2 + g(2, 4, 2; 1, 2, 1) = 364.286$
 $g(4, 4, 2; 2, 2, 1) = 100/2 + g(2, 5, 2; 1, 2, 1) = 284.524$
 $g(7, 4, 2; 2, 2, 1) = 100/5 + g(5, 5, 2; 2, 2, 1) = 329.524$
 $g(8, 3, 2; 2, 2, 1) = 100/6 + g(6, 4, 2; 1, 2, 1) = 359.524$
 $g(8, 3, 4; 2, 2, 1) = 100/6 + g(6, 4, 5; 1, 2, 1) = 340.953$
 $g(4, 2, 5; 2, 2, 1) = 100/2 + g(2, 2, 6; 1, 2, 1) = 272.143$
 $g(4, 2, 4; 2, 2, 1) = 100/2 + g(2, 3, 5; 1, 2, 1) = 351.190$
 $g(2, 4, 2; 2, 2, 1) = 200/2 + g(4, 2, 4; 1, 2, 1) = 301.190$
 $g(3, 4, 2; 2, 2, 1) = 200/2 + g(5, 2, 4; 2, 1, 1) = 347.619$
 $g(7, 4, 2; 2, 2, 1) = 200/2 + g(9, 2, 2; 2, 1, 1) = 407.143$
 $g(8, 4, 2; 2, 2, 1) = 200/2 + g(10, 2, 4; 2, 1, 1) = 355.357$
 $g(8, 4, 4; 2, 2, 1) = 200/2 + g(10, 2, 8; 2, 1, 1) = 335.397$
 $g(4, 4, 2; 2, 2, 1) = 200/2 + g(6, 3, 2; 2, 1, 1) = 362.857$
 $g(2, 5, 2; 2, 2, 1) = 200/2 + g(2, 3, 3; 2, 1, 1) = 334.523$
 $g(2, 4, 2; 2, 2, 1) = 200/2 + g(2, 2, 5; 2, 1, 1) = 326.619$
 $g(2, 2, 5; 2, 2, 1) = 300/2 + g(11, 6, 2; 2, 2, 0) = 264.286$

In the above calculations, dominated states are omitted, and the optimal sequence of decisions is highlighted. The optimal cost found at the fifth job choice is 264.286 at the state (2, 2, 5; 2, 2, 1). We backtrack from this solution to identify the optimal sequence of solutions recursively, as follows: (2, 2, 5; 2, 2, 1) $\rightarrow$ (11, 6, 2; 2, 2, 0) $\rightarrow$ (10, 7, 2; 1, 2, 0) $\rightarrow$ (12, 5, 2; 1, 1, 0) $\rightarrow$ (6, 8, 5; 0, 1, 0) $\rightarrow$ (7, 7, 7; 0, 0, 0).

Consequently, the optimal sequence of jobs, processing times, and current makespan values is: job (2, 1) requires time 14.286; job (1, 1)

requires time 50, and finishes at time 14.286 + 50 = 64.286; job (2, 2) requires time 40, and finishes at time 64.286 + 40 = 104.286; job (1, 2) requires time 10 and finishes at time 104.286 + 10 = 114.286; and job (3, 1) requires time 150 and finishes at time 114.286 + 150 = 264.286.

A.3. Numerical example of Algorithm GH4

Example 3. Data: $m = 3; n = 10; n_1 = 4, n_2 = 2, n_3 = 4$: $p_{1,1} = 37, p_{1,2} = 7, p_{1,3} = 34, p_{1,4} = 31, p_{2,1} = 10, p_{2,2} = 19, p_{3,1} = 23, p_{3,2} = 23, p_{3,3} = 28, p_{3,4} = 22; L_j = 2, U_j = 20, v_j^0 = 7, j = 1, 2, 3; l_1 = .758, l_2 = .773, l_3 = .623; f_{12} = .042, f_{13} = .037, f_{21} = .023, f_{23} = .030, f_{31} = .018, f_{32} = .032$. We use $\phi_j(v_j^*, x_j) = p_{j,x_j} * (v_j^0/v_j^*)^{\alpha_{1j}} * (\sum_{i=1}^m x_i)^{\alpha_{2j}}$, where $\alpha_{1j} = \{0.931, 0.936, 0.965\}$, and $\alpha_{2j} = \{-0.042, -0.021, -0.074\}$, for $j = 1, 2, 3$. The solution is as follows:

Step 1. For all job types with unscheduled jobs, calculate the ratio of the learning increment of the next job of each type to the total forgetting decrement of the jobs of the other types.

Job 1 of Type 1: $\delta_l^+(1, 1) = p_{1,2} \times (\frac{1}{v_1^0} - \frac{1}{\min\{U_1, v_1^0 + l_1 \times p_{1,1}\}}) = 7 \times (\frac{1}{7} - \frac{1}{\min\{20, 7+0.758 \times 37\}}) = 0.65$; $\delta_f^-(1, 1) = (p_{2,1} + p_{2,2}) \times (\frac{1}{\max\{v_2^0 - f_{12} \times p_{1,1}, L_2\}} - \frac{1}{v_2^0}) + (p_{3,1} + p_{3,2} + p_{3,3} + p_{3,4}) \times (\frac{1}{\max\{v_3^0 - f_{13} \times p_{1,1}, L_3\}} - \frac{1}{v_3^0}) = (10 + 19) \times (\frac{1}{\max\{7-0.042 \times 37, 2\}} - \frac{1}{7}) + (23 + 23 + 28 + 22) \times (\frac{1}{\max\{7-0.037 \times 37, 2\}} - \frac{1}{7}) = 3.333$; $ratio = \delta_l^+(1, 1)/\delta_f^-(1, 1) = 0.191$.

Job 1 of Type 2: $\delta_l^+(2, 1) = p_{2,2} \times (\frac{1}{v_2^0} - \frac{1}{\min\{U_2, v_2^0 + l_2 \times p_{2,1}\}}) = 19 \times (\frac{1}{7} - \frac{1}{\min\{20, 7+0.773 \times 10\}}) = 1.424$; $\delta_f^-(2, 1) = (p_{1,1} + p_{1,2} + p_{1,3} + p_{1,4}) \times (\frac{1}{\max\{v_1^0 - f_{21} \times p_{2,1}, L_1\}} - \frac{1}{v_1^0}) + (p_{3,1} + p_{3,2} + p_{3,3} + p_{3,4}) \times (\frac{1}{\max\{v_3^0 - f_{23} \times p_{2,1}, L_3\}} - \frac{1}{v_3^0}) = (37 + 7 + 34 + 31) \times (\frac{1}{\max\{7-0.023 \times 10, 2\}} - \frac{1}{7}) + (23 + 23 + 28 + 22) \times (\frac{1}{\max\{7-0.030 \times 10, 2\}} - \frac{1}{7}) = 0.589$; $ratio = \delta_l^+(2, 1)/\delta_f^-(2, 1) = 1.369$.

Job 1 of Type 3: $\delta_l^+(3, 1) = p_{3,2} \times (\frac{1}{v_3^0} - \frac{1}{\min\{U_3, v_3^0 + l_3 \times p_{3,1}\}}) = 23 \times (\frac{1}{7} - \frac{1}{\min\{20, 7+0.623 \times 23\}}) = 2.135$; $\delta_f^-(3, 1) = (p_{1,1} + p_{1,2} + p_{1,3} + p_{1,4}) \times (\frac{1}{\max\{v_1^0 - f_{31} \times p_{3,1}, L_1\}} - \frac{1}{v_1^0}) + (p_{2,1} + p_{2,2}) \times (\frac{1}{\max\{v_2^0 - f_{32} \times p_{3,1}, L_2\}} - \frac{1}{v_2^0}) = (37 + 7 +$

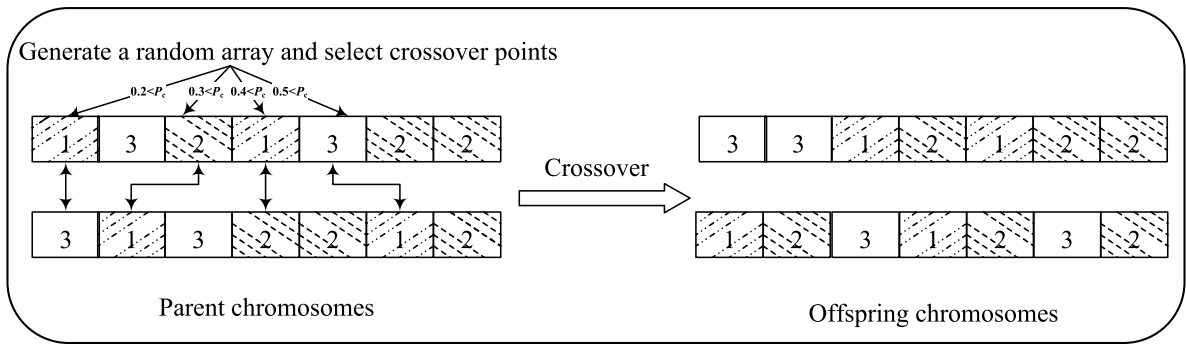


Fig. 4. Crossover.

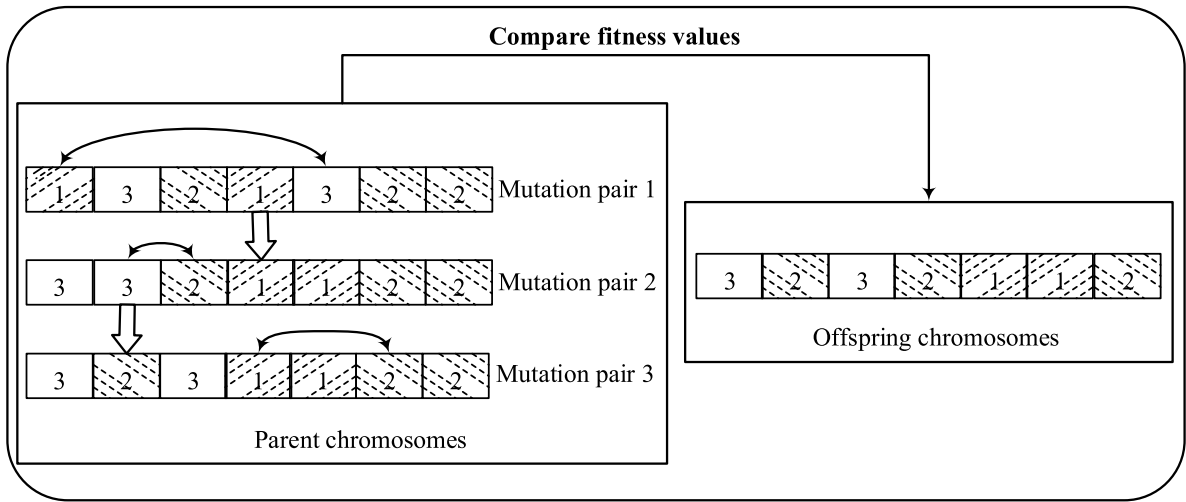


Fig. 5. Mutation.

$34 + 31) \times (\frac{1}{\max\{7-0.018 \times 23, 2\}} - \frac{1}{7}) + (10 + 19) \times (\frac{1}{\max\{7-0.032 \times 23, 2\}} - \frac{1}{7}) = 0.511$;
 $ratio = \delta_f^+(3, 1) / \delta_f^-(3, 1) = 0.192$.

Therefore, choose job 1 of Type 2 with maximum ratio to schedule first. After processing job 1 of Type 2, the learning levels of the three job types are $v'_1 = \max\{v_1^0 - f_{21} \times p_{2,1}, L_1\} = 6.77$, $v'_2 = \min\{U_2, v_2^0 + l_2 \times p_{2,1}\} = 14.73$, and $v'_3 = \max\{v_3^0 - f_{23} \times p_{2,1}, L_3\} = 6.7$.

Next, there is only one job left of Type 2. Therefore, compare the forgetting decrements and select a type with smallest forgetting decrement for processing.

Job 1 of Type 1: $\delta_f^-(1, 1) = p_{2,2} \times (\frac{1}{\max\{v'_2 - f_{12} \times p_{1,1}, L_2\}} - \frac{1}{v'_2}) + (p_{3,1} + p_{3,2} + p_{3,3} + p_{3,4}) \times (\frac{1}{\max\{v'_3 - f_{13} \times p_{1,1}, L_3\}} - \frac{1}{v'_3}) = 19 \times (\frac{1}{\max\{14.73 - 0.042 \times 37.2\}} - \frac{1}{14.73}) + (23 + 23 + 28 + 22) \times (\frac{1}{\max\{6.7 - 0.037 \times 37.2\}} - \frac{1}{6.7}) = 3.775$;

Job 2 of Type 2: $\delta_f^-(2, 2) = (p_{1,1} + p_{1,2} + p_{1,3} + p_{1,4}) \times (\frac{1}{\max\{v'_1 - f_{21} \times p_{2,2}, L_1\}} - \frac{1}{v'_1}) + (p_{3,1} + p_{3,2} + p_{3,3} + p_{3,4}) \times (\frac{1}{\max\{v'_3 - f_{23} \times p_{2,2}, L_3\}} - \frac{1}{v'_3}) = (37 + 7 + 34 + 31) \times (\frac{1}{\max\{6.77 - 0.023 \times 19.2\}} - \frac{1}{6.77}) + (23 + 23 + 28 + 22) \times (\frac{1}{\max\{6.7 - 0.030 \times 19.2\}} - \frac{1}{6.7}) = 2.368$;

Job 1 of Type 3: $\delta_f^-(3, 1) = (p_{1,1} + p_{1,2} + p_{1,3} + p_{1,4}) \times (\frac{1}{\max\{v'_1 - f_{31} \times p_{3,1}, L_1\}} - \frac{1}{v'_1}) + p_{2,2} \times (\frac{1}{\max\{v'_2 - f_{32} \times p_{3,1}, L_2\}} - \frac{1}{v'_2}) = (37 + 7 + 34 + 31) \times (\frac{1}{\max\{6.77 - 0.018 \times 23, 2\}} - \frac{1}{6.77}) + 19 \times (\frac{1}{\max\{14.73 - 0.032 \times 23, 2\}} - \frac{1}{14.73}) = 1.207$;

Therefore, choose job 1 of Type 3 with smallest forgetting decrement for processing. After processing job 1 of Type 3, the learning levels of the three job types are $v'_1 = 6.356$, $v'_2 = 13.994$, and $v'_3 = 20$. Continuing this process, we obtain the initial sequence: $(2, 1) \rightarrow (3, 1) \rightarrow (1, 1) \rightarrow (2, 2) \rightarrow (3, 2) \rightarrow (1, 2) \rightarrow (3, 3) \rightarrow (3, 4) \rightarrow (1, 3) \rightarrow (1, 4)$. The learning levels of each job type $T_j, j \in \{1, 2, 3\}$, are shown in Table 7, and the makespan is 131.475.

Finally, we implement local search (see Algorithm 4) by swapping all adjacent pairs of jobs of different types, and output the resulting schedule with the optimal makespan, i.e., 131.307. The optimal sequence is $(2, 1) \rightarrow (3, 1) \rightarrow (1, 1) \rightarrow (2, 2) \rightarrow (3, 2) \rightarrow (3, 3) \rightarrow (3, 4) \rightarrow (1, 2) \rightarrow (1, 3) \rightarrow (1, 4)$.

A.4. Numerical example of Algorithm GA

We consider an instance comprising 7 jobs distributed across three types, with the number of jobs in each type being 2, 3 and 2, respectively. We use the encoding method where each integer represents a specific job type, as shown in Fig. 2. We then provide the crossover and mutation operators of GA. In the crossover operation, two individuals (referred to as Parent 1 ($\{T_1, T_3, T_2, T_1, T_3, T_2, T_2\}$) and Parent 2 ($\{T_3, T_1, T_3, T_2, T_2, T_1, T_2\}$)) are randomly selected from the population to serve as parents. A one-dimensional random array, such as $[0.2, 0.9, 0.3, 0.4, 0.5, 0.8, 0.8]$, is generated for Parent 1 to determine crossover points. The crossover rate is set at $P_c = 0.7$, meaning that genes corresponding to array values less than 0.7 are selected for crossover. We let $k_3 = 0.1$ and $k_4 = 0.2$. For the example shown in Fig. 4, the selected crossover genes from Parent 1 are at positions 1, 3, 4, and 5. The corresponding crossover genes in Parent 2 are then identified based on the sequence of selected genes in Parent 1, which for this instance are located at positions 1, 2, 4, and 6. To ensure the consistency of exchanged genetic material, the sets of crossover genes from both parents are aligned. The selected genes are then swapped between Parent 1 and Parent 2 in their respective positions. This process facilitates the creation of offspring that inherit genetic traits from both parents.

Table 7
Learning level changes of each job type T_j .

	$P_{2,1}$	$P_{3,1}$	$P_{1,1}$	$P_{2,2}$	$P_{3,2}$	$P_{1,2}$	$P_{3,3}$	$P_{3,4}$	$P_{1,3}$	$P_{1,4}$
T_1	6.770	6.356	20.000	19.563	19.149	20.000	19.496	19.100	20.000	20.000
T_2	14.730	13.994	12.440	20.000	20.000	20.000	20.000	20.000	20.000	20.000
T_3	6.700	20.000	18.631	18.061	20.000	19.741	20.000	20.000	20.000	20.000

The probability of mutation for each individual is dependent on its objective value. Let the maximum objective value be $\gamma_{\max} = 200$ and the average objective value be $\gamma_{\text{avg}} = 100$. For example, if the objective value of individual 1 is $\gamma_1 = 80$, the mutation probability is calculated as $P_m = k_4 = 0.2$. If the objective value of individual 1 is $\gamma_1 = 120$, the mutation probability becomes $P_m = k_3 \left(\frac{\gamma_{\max} - \gamma_1}{\gamma_{\max} - \gamma_{\text{avg}}} \right) = 0.08$. After determining the mutation probability, a random real number r is generated from the range $[0,1]$. If $r \leq P_m$, three genes are randomly selected from the individual, and these genes undergo exchange to produce a new offspring, as shown in Fig. 5. This mutation process contributes to the search for better solutions by introducing variations that may lead to improved offspring performance.

References

Abreu, L. R., Cunha, J. O., Prata, B. A., & Framinan, J. M. (2020). A genetic algorithm for scheduling open shops with sequence-dependent setup times. *Computers & Operations Research*, 113, Article 104793.

Afshari, H., Jaber, M. Y., & Searcy, C. (2019). Investigating the effects of learning and forgetting on the feasibility of adopting additive manufacturing in supply chains. *Computers & Industrial Engineering*, 128, 576–590.

Akbari Arekamari, M., & Ghasemi Namaghi, M. (2023). Optimizing human reliability in work shifts scheduling: A real case of clothing job shop. *Quality and Reliability Engineering International*, 39(8), 3518–3536.

Asadayoobi, N., Taghipour, S., & Jaber, M. (2023). Optimising stochastic task allocation and scheduling plans for mission workers subject to learning-forgetting, fatigue-recovery, and stress-recovery effects. *Expert Systems with Applications*, 229, Article 120524.

Asghari, M., Afshari, H., Jaber, M., & Searcy, C. (2024). Learning and forgetting interactions within a collaborative human-centric manufacturing network. *European Journal of Operational Research*, 313(3), 977–991.

Bailey, C. D. (1989). Forgetting and the learning curve: A laboratory study. *Management Science*, 35(3), 340–352.

Borjonon, T., & Maenhout, B. (2024). The impact of dynamic learning and training on the personnel staffing decision. *Computers & Industrial Engineering*, 187, Article 109784.

Cheng, R., Gen, M., & Tsujimura, Y. (1999). A tutorial survey of job-shop scheduling problems using genetic algorithms, Part II: Hybrid genetic search strategies. *Computers & Industrial Engineering*, 36(2), 343–364.

Chu, X., Gao, D., Cheng, S., Wu, L., Chen, J., Shi, Y., & Qin, Q. (2019). Worker assignment with learning-forgetting effect in cellular manufacturing system using adaptive memetic differential search algorithm. *Computers & Industrial Engineering*, 136, 381–396.

Frank, A. G., Dalenogare, L. S., & Ayala, N. F. (2019). Industry 4.0 technologies: Implementation patterns in manufacturing companies. *International Journal of Production Economics*, 210, 15–26.

Geurtsen, M., Didden, J. B., Adan, J., Atan, Z., & Adan, I. (2023). Production, maintenance and resource scheduling: A review. *European Journal of Operational Research*, 305(2), 501–529.

Haunschild, P. R., Polidoro, F., & Chandler, D. (2015). Organizational oscillation between learning and forgetting: The dual role of serious errors. *Organization Science*, 26(6), 1682–1701.

Heuser, P., & Tauer, B. (2023). Single-machine scheduling with product category-based learning and forgetting effects. *Omega*, 115, Article 102786.

Hoedt, S. (2019). The effect of job similarity on forgetting in multi-task production. *Procedia Manufacturing*, 39, 983–990.

Howick, S., & Eden, C. (2007). Learning in disrupted projects: on the nature of corporate and personal learning. *International Journal of Production Research*, 45(12), 2775–2797.

Hwang, F. J., Kovalyov, M. Y., & Lin, B. M. T. (2012). Total completion time minimization in two-machine flow shop scheduling problems with a fixed job sequence. *Discrete Optimization*, 9(1), 29–39.

Hwang, F. J., Kovalyov, M. Y., & Lin, B. M. T. (2014). Scheduling for fabrication and assembly in a two-machine flowshop with a fixed job sequence. *Annals of Operations Research*, 217, 263–279.

Hwang, F. J., & Lin, B. M. T. (2011). Coupled-task scheduling on a single machine subject to a fixed-job-sequence. *Computers & Industrial Engineering*, 60(4), 690–698.

Jaber, M. Y. (2006). Learning and forgetting models and their applications. In *Handbook of industrial and systems engineering*: vol. 30, (no. 1), (pp. 30–127).

Jaber, M. Y., Kher, H. V., & Davis, D. J. (2003). Countering forgetting through training and deployment. *International Journal of Production Economics*, 85(1), 33–46.

Jaber, M., Peltokorpi, J., Glock, C., Grosse, E., & Pusic, M. (2021). Adjustment for cognitive interference enhances the predictability of the power learning curve. *International Journal of Production Economics*, 234, Article 108045.

Kar, S., Castellucci, G., Filice, S., Malmasi, S., & Rokhlenko, O. (2022). Preventing catastrophic forgetting in continual learning of new natural language tasks. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 3137–3145).

Karhi, S., & Shabtay, D. (2014). Online scheduling of two job types on a set of multipurpose machines. *International Journal of Production Economics*, 150, 155–162.

Kogan, K., El Ouardighi, F., & Herbon, A. (2017). Production with learning and forgetting in a competitive environment. *International Journal of Production Economics*, 189, 52–62.

Korytkowski, P. (2017). Competences-based performance model of multi-skilled workers with learning and forgetting. *Expert Systems with Applications*, 77, 226–235.

Kouteck, M., & Zink, J. (2020). Complexity of scheduling few types of jobs on related and unrelated machines. Working paper. Available at: <http://dx.doi.org/10.48550/arXiv.2009.11840>.

Liang, Z., Liu, M., Zhong, P., & Zhang, C. (2023). Application research of a new neighbourhood structure with adaptive genetic algorithm for job shop scheduling problem. *International Journal of Production Research*, 61(2), 362–381.

Lin, B. M. T., & Hwang, F. J. (2011). Total completion time minimization in a 2-stage differentiation flowshop with fixed sequences per job type. *Information Processing Letters*, 111(5), 208–212.

Mustu, S., & Eren, T. (2021). The single machine scheduling problem with setup times under an extension of the general learning and forgetting effects. *Optimization Letters*, 15(4), 1327–1343.

Ostermeier, F. F. (2020). The impact of human consideration, schedule types and product mix on scheduling objectives for unpaced mixed-model assembly lines. *International Journal of Production Research*, 58(14), 4386–4405.

Ostermeier, F. F., & Deuse, J. (2024). Modelling forgetting due to intermittent production in mixed-model line scheduling. *Flexible Services and Manufacturing Journal*, 36(2), 503–532.

Pan, E., Wang, G., Xi, L., Chen, L., & Han, X. (2014). Single-machine group scheduling problem considering learning, forgetting effects and preventive maintenance. *International Journal of Production Research*, 52(19), 5690–5704.

Pei, J., Zhou, Y., Yan, P., & Pardalos, P. M. (2023). A concise guide to scheduling with learning and deteriorating effects. *International Journal of Production Research*, 61(6), 2010–2031.

Qaraad, M., Amjad, S., Hussein, N. K., Farag, M., Mirjalili, S., & Elhosseini, M. A. (2024). Quadratic interpolation and a new local search approach to improve particle swarm optimization: Solar photovoltaic parameter estimation. *Expert Systems with Applications*, 236, Article 121417.

Ramdas, K., Saleh, K., Stern, S. N., & Liu, H. (2018). Variety and experience: Learning and forgetting in the use of surgical devices. *Management Science*, 64(6), 2590–2608.

Shabtay, D., & Karhi, S. (2012). Online scheduling of two job types on a set of multipurpose machines with unit processing times. *Computers & Operations Research*, 39(2), 405–412.

Srinivas, M., & Patnaik, L. M. (1994). Adaptive probabilities of crossover and mutation in genetic algorithms. *IEEE Transactions on Systems, Man and Cybernetics*, 24(4), 656–667.

Teyarachakul, S., Chand, S., & Ward, J. (2011). Effect of learning and forgetting on batch sizes. *Production and Operations Management*, 20(1), 116–128.

Van Der Veen, J. A., & Zhang, S. (1996). Low-complexity algorithms for sequencing jobs with a fixed number of job-classes. *Computers & Operations Research*, 23(11), 1059–1067.

Wang, J., Liu, C., & Zhou, M. C. (2020). Improved bacterial foraging algorithm for cell formation and product scheduling considering learning and forgetting factors in cellular manufacturing systems. *IEEE Systems Journal*, 14(2), 3047–3056.

- Wright, T. P. (1936). Factors affecting the cost of airplanes. *Journal of the Aeronautical Sciences*, 3(4), 122–128.
- Wu, J., Song, B., Zhang, G., Ou, J., Chen, Y., Yao, F., He, L., & Xing, L. (2022). A data-driven improved genetic algorithm for agile earth observation satellite scheduling with time-dependent transition time. *Computers & Industrial Engineering*, 174, Article 108823.
- Yang, W. H., & Chand, S. (2008). Learning and forgetting effects on a group scheduling problem. *European Journal of Operational Research*, 187(3), 1033–1044.
- Yin, Y., Wu, C.-C., Wu, W.-H., & Chen, J.-C. (2013). Single-machine group scheduling with a general learning effect. *European Journal of Industrial Engineering*, 7(3), 350–369.
- Zhang, L., Deng, Q., Lin, R., Gong, G., & Han, W. (2021). A combinatorial evolutionary algorithm for unrelated parallel machine scheduling problem with sequence and machine-dependent setup times, limited worker resources and learning effect. *Expert Systems with Applications*, 175, Article 114843.